

INNOPOLIS UNIVERSITY

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 12 · Стерео и эпиполярная геометрия

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Карта курса

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография и масштабирование изображение	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	**L12 Стерео и эпиполярная геометрия**	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L10: Гомография: оценивание и выравнивание

1 Стереозрение

Термины: Стереозрение (Stereo vision) · Триангуляция · Трилатерация · Диспаратность (Disparity) · Эпиполярное ограничение (Epipolar constraint) · Эпиполярная ректификация

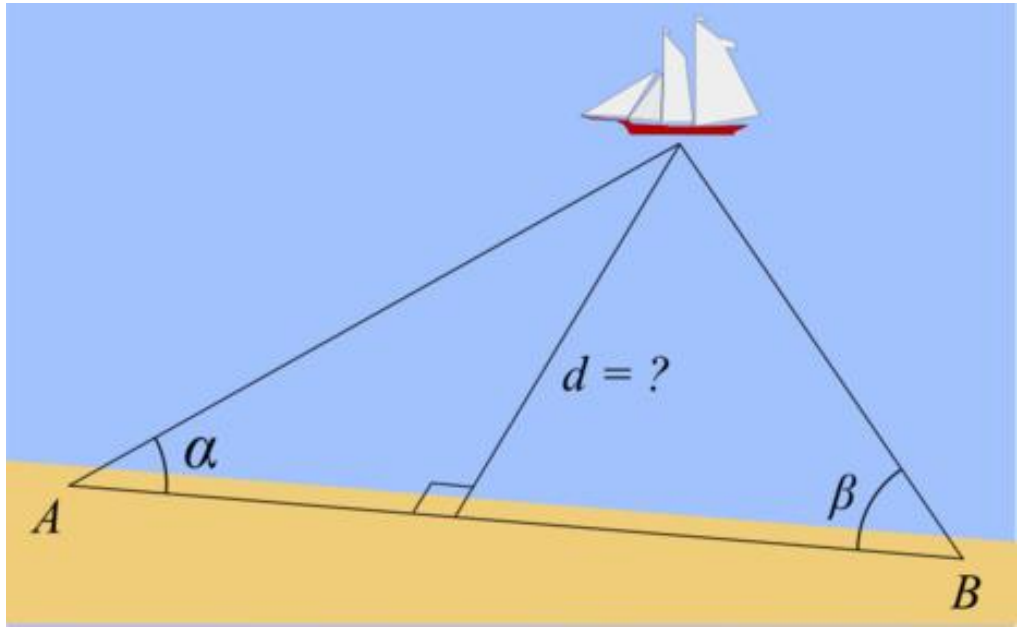
Triangulation

Триангуляция — вычисление координат удаленной точки (или расстояния *d*) через измерение углов (α и β) из двух известных точек с фиксированным базовым расстоянием (l) с использованием закона синусов.

In trigonometry, **triangulation** is the process of determining the **point location** by measuring **angles** to its position from known points at either end of a fixed **baseline**.

Triangulation may be used to calculate the coordinates and distance from the shore to a ship. If we measure the angles α and β between the shore and the ship, and know the distance *l* between A and B or the coordinates of A and B, then the **law of sines** can be applied to find the coordinates of the ship and the distance d.

In geometry, **trilateration** is defined as the process of determining absolute or relative **locations of points by measurement of distances**, using the geometry of circles, spheres or triangles.



$$d = \ell \, \frac{\sin\alpha \sin\beta}{\sin(\alpha + \beta)}$$

Fig. © Triangulation, Wikipedia

How do we measure distances with cameras?

- **Стереозрение:** восстановление 3D-сцены по двум камерам с известным взаимным расположением (*T*) и ориентацией (*R*).
- **Structure from Motion (SfM):** восстановление 3D-структуры и параметров движения камер (с точностью до масштаба) по набору кадров, когда параметры камер изначально могут быть неизвестны.

- **Stereo vision:**
  using 2 cameras with **known** relative position T and orientation R, recover the **3D scene information**



- **Structure from Motion:**
  recover the **3D scene structure & the camera poses** (up to scale) from multiple images, from potentially **unknown** cameras

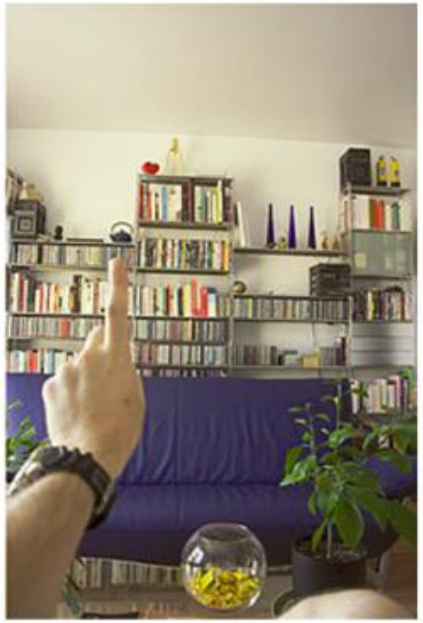
Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Disparity in the human retina

Биологический аналог стереозрения. Мозг человека объединяет два слегка смещенных изображения с левой и правой сетчатки глаза в одну 3D-картину, используя **диспаратность** (смещение/разность координат точек на изображениях).

- **Stereopsys:** the brain allows us to see the left and right retinal images as a single 3D image
- Observe ***image disparity***





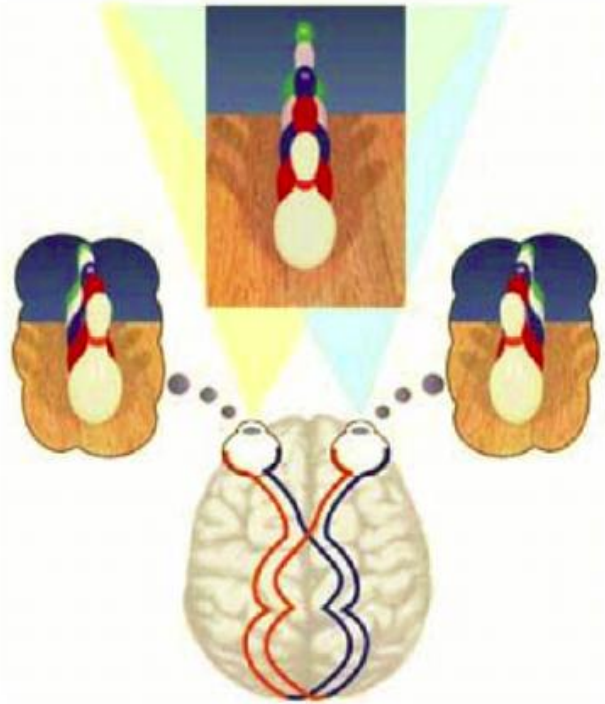


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

How do we measure distances with cameras?

Проблема: один снимок дает лишь один ракурс объекта, но не его глубину.
Для восстановления 3D-структуры объекта необходимо наблюдать сцену с двух разных ракурсов и находить точку пересечения лучей.

- From a single image: we can only deduct the **ray** along which each image-point lies

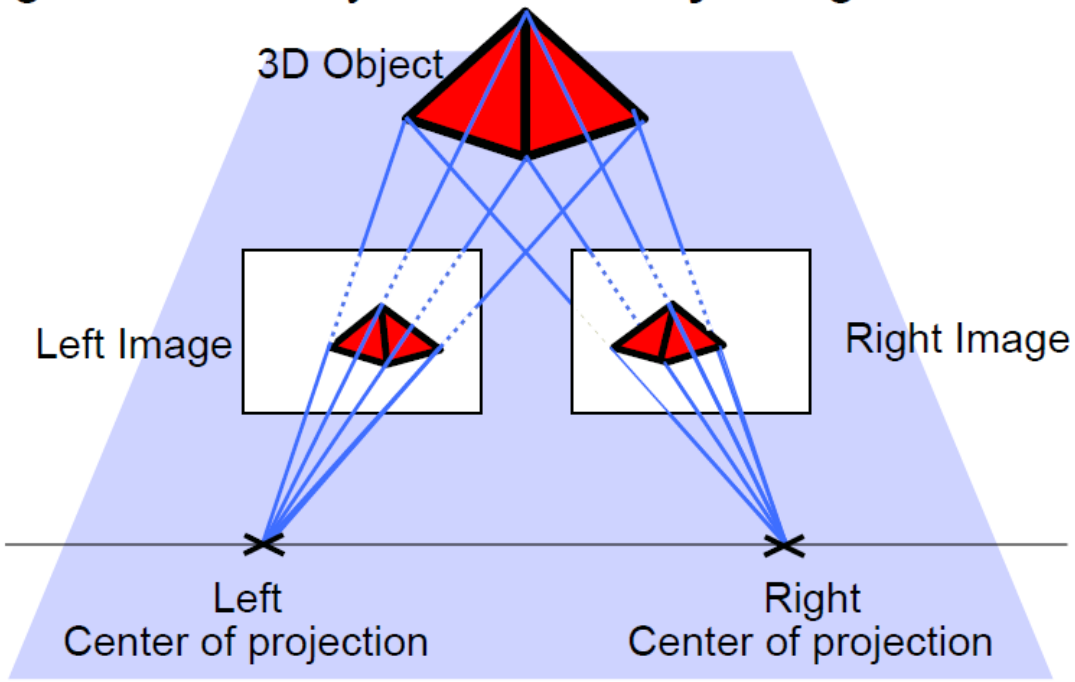


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

- Observe the scene from 2 different viewpoints ⇨ solve for the intersection of the rays and recover the 3D structure

Stereo Vision | simplified case

Математика идеального случая. Две одинаковые камеры выровнены по оси *X*. Через подобие треугольников выводится формула глубины: $Z_p = b\frac{f}{u_l - u_r}$, где ключевыми факторами являются **база** (b — расстояние между камерами) и **диспаратность** ($u_l - u_r$) — разность координат точки на левом и правом кадрах).

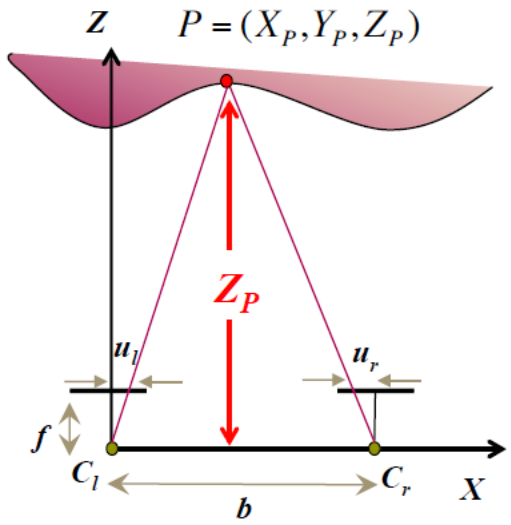
- An ideal, simplified case: assume both cameras are **identical** and are **aligned** with the x-axis

From Similar Triangles:

$$\frac{f}{Z_P} = \frac{u_l}{X_P}$$
$$\frac{f}{Z_P} = \frac{u_r}{b - X_P}$$

$$Z_P = \frac{bf}{u_l - u_r}$$

Disparity
difference in image location of the projection of a 3D point in two image planes



Baseline
distance between the optical centers of the two cameras

Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Stereo Vision | disparity map

Карта диспаратности (смещений). Для каждого пикселя левого кадра ищется пара на правом, и вычисляется разность их координат. Ближние объекты имеют большее смещение (диспаратность), что наглядно иллюстрируется яркостью на карте.

- **Disparity:** the difference in image location of corresponding 3D points as projected to the left & right images
- **Disparity map** holds the disparity value at every pixel:
  - Identify correspondent points of **all image pixels** in the original images
  - Compute the disparity for each pair of correspondences
- Near-by objects experience bigger disparity:

$$Z_P = \frac{bf}{u_l - u_r}$$



Left image



Right image



Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Stereo Vision | general case

Реальные условия. Идеально выровненных и одинаковых камер не существует. Для работы настоящей стереосистемы необходимо находить **внешние параметры** (взаимное положение камер (*R, T*) и **внутренние параметры** каждой камеры (фокусное расстояние, оптический центр, дисторсию) с помощью методов калибровки.

- Two identical cameras do not exist in nature!
- Aligning both cameras on a horizontal axis is very difficult
- In order to use a stereo camera, we need:
  - **relative pose** between the cameras (rotation, translation), and ⇨ "extrinsics"
  - the **focal length**, **optical center**, **radial distortion** of each ⇨ "intrinsics"

⇨ Use calibration method

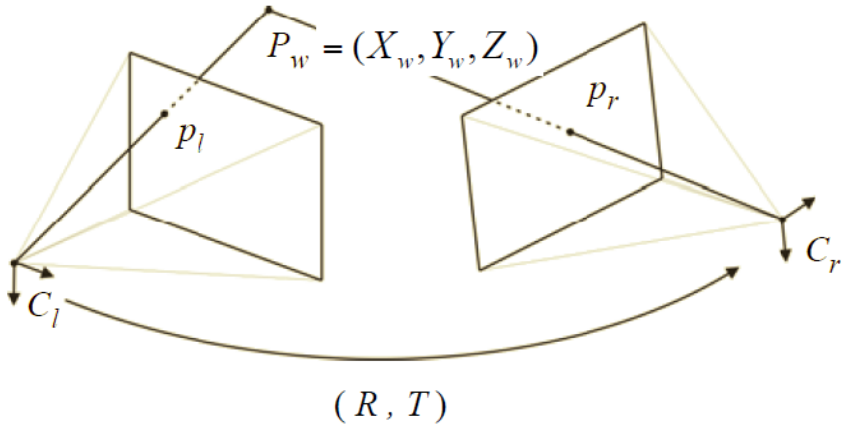


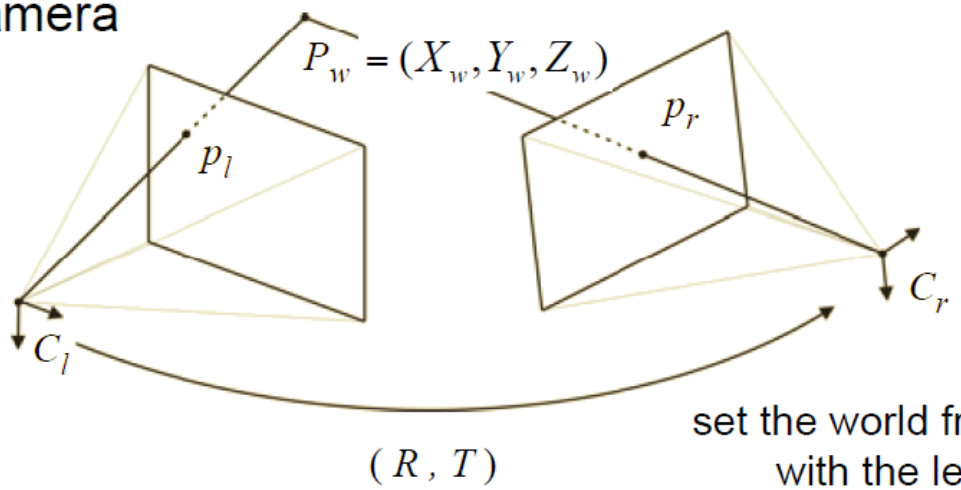


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Stereo Vision | general case

Координаты 3D-точки определяются через систему уравнений проекции для левой и правой камер. Мировая система координат привязывается к левой камере, а правая смещена на параметры калибровки *(R, T)*.

- To estimate the 3D position of P_w we just construct the system of equations of the left and right camera



Left camera: set the world frame to coincide with the left camera frame

$$\tilde{p}_l = \lambda_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = K_l \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix}$$

- Triangulation: the problem of determining the 3D position of a point, given a set of *corresponding* image locations & known camera poses.

Right camera:

$$\tilde{p}_r = \lambda_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = K_r R \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + T$$

© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Binocular Stereo

Базовая геометрическая схема бинокулярного стереозрения показывает, как лучи от одной точки сцены проецируются через оптические центры на две разные плоскости изображений.

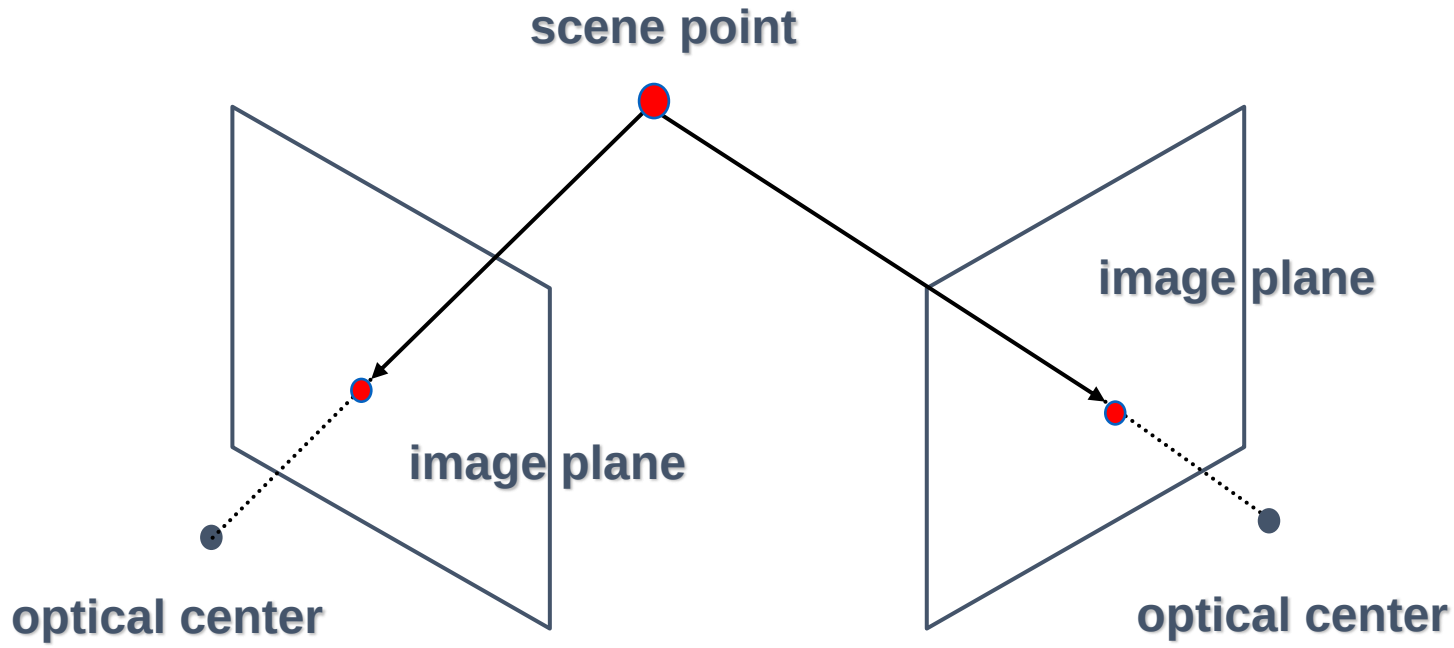


Fig. © S. Narasimhan, Computer Vision, CS, CMU, 2010

Binocular Stereo

Базовые требования. Триангуляция (как пересечение двух лучей) требует двух обязательных условий: точной **калибровки** камер и нахождения **точек соответствия** (corresponding points) на левом и правом кадрах.

- Basic Principle: Triangulation
  - Gives reconstruction as intersection of two rays
  - Requires:
    - calibration
    - point correspondence

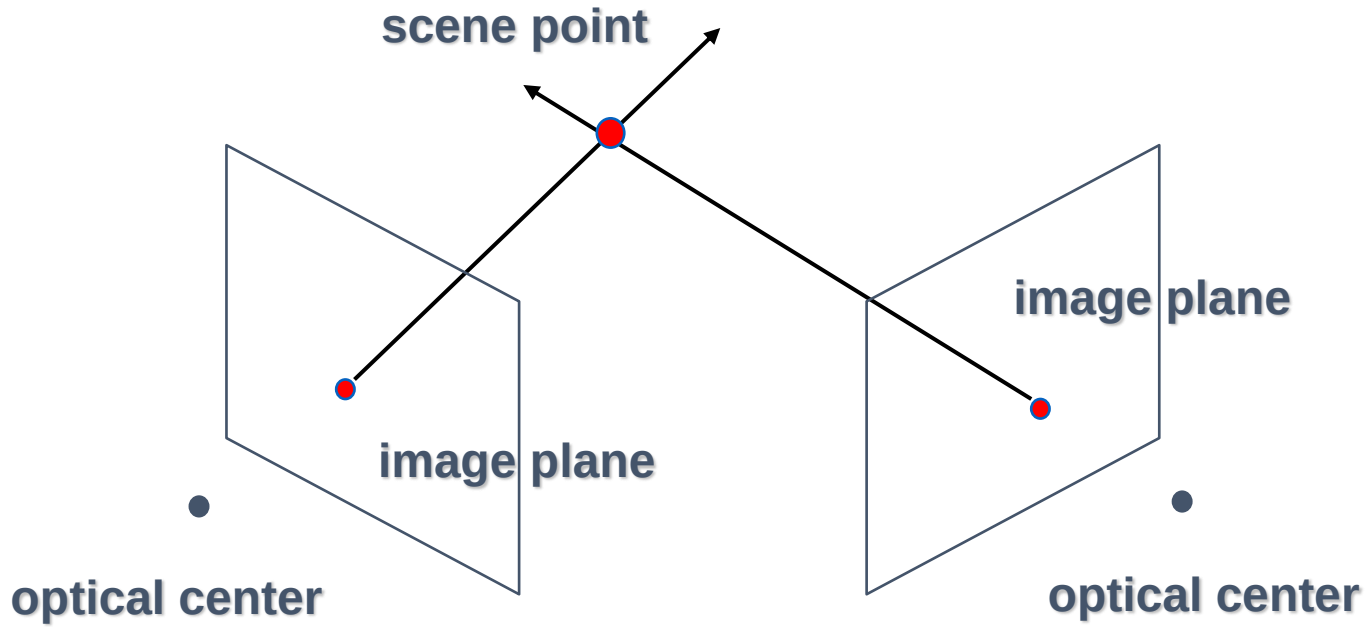
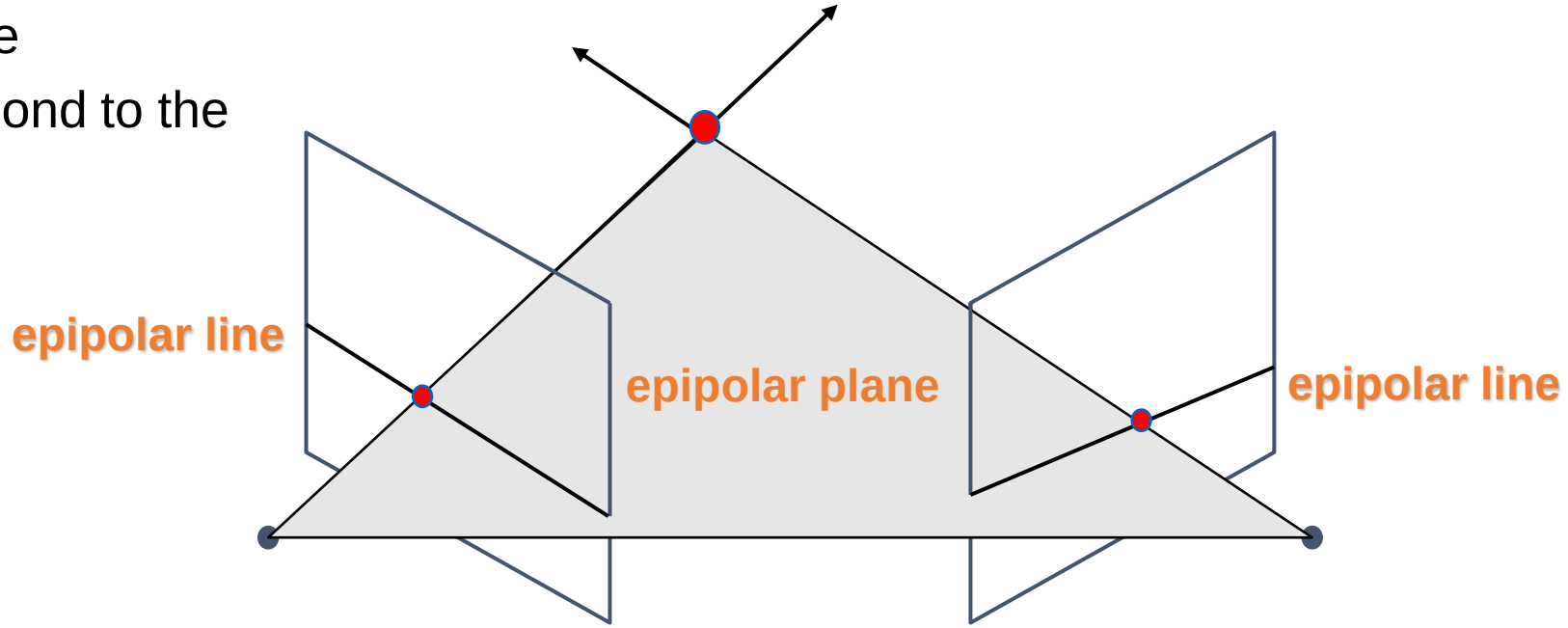


Fig. © S. Narasimhan, Computer Vision, CS, CMU, 2010

Stereo Correspondence

Эпиполярное ограничение (Epipolar Constraint). Формулируется главное решение проблемы поиска соответствий: точка, её проекции и оптические центры лежат в одной **эпиполярной плоскости**. Это сужает сложную задачу поиска парного пикселя по всему 2D-изображению до простого **1D-поиска вдоль сопряженной эпиполярной линии**.

- Determine Pixel Correspondence
  - Pairs of points that correspond to the same scene point



- Epipolar Constraint
  - Reduces correspondence problem to 1D search along *conjugate epipolar lines*

Fig. © S. Narasimhan, Computer Vision, CS, CMU, 2010

Stereo Image Rectification

Ректификация изображений — процесс проецирования исходных кадров на общую виртуальную плоскость так, чтобы соответствующие точки сцены получили одинаковую координату строки (*Y*). Это преобразует сложный 2D-поиск парных пикселей в простой 1D-поиск.

Stereo image rectification projects images onto a common image plane in such a way that the corresponding points have the same row coordinates. This process is useful for stereo vision, because the 2-D stereo correspondence problem reduces to a 1-D problem.

As an example, stereo image rectification is often used as a pre-processing step for computing disparity or creating anaglyph images.

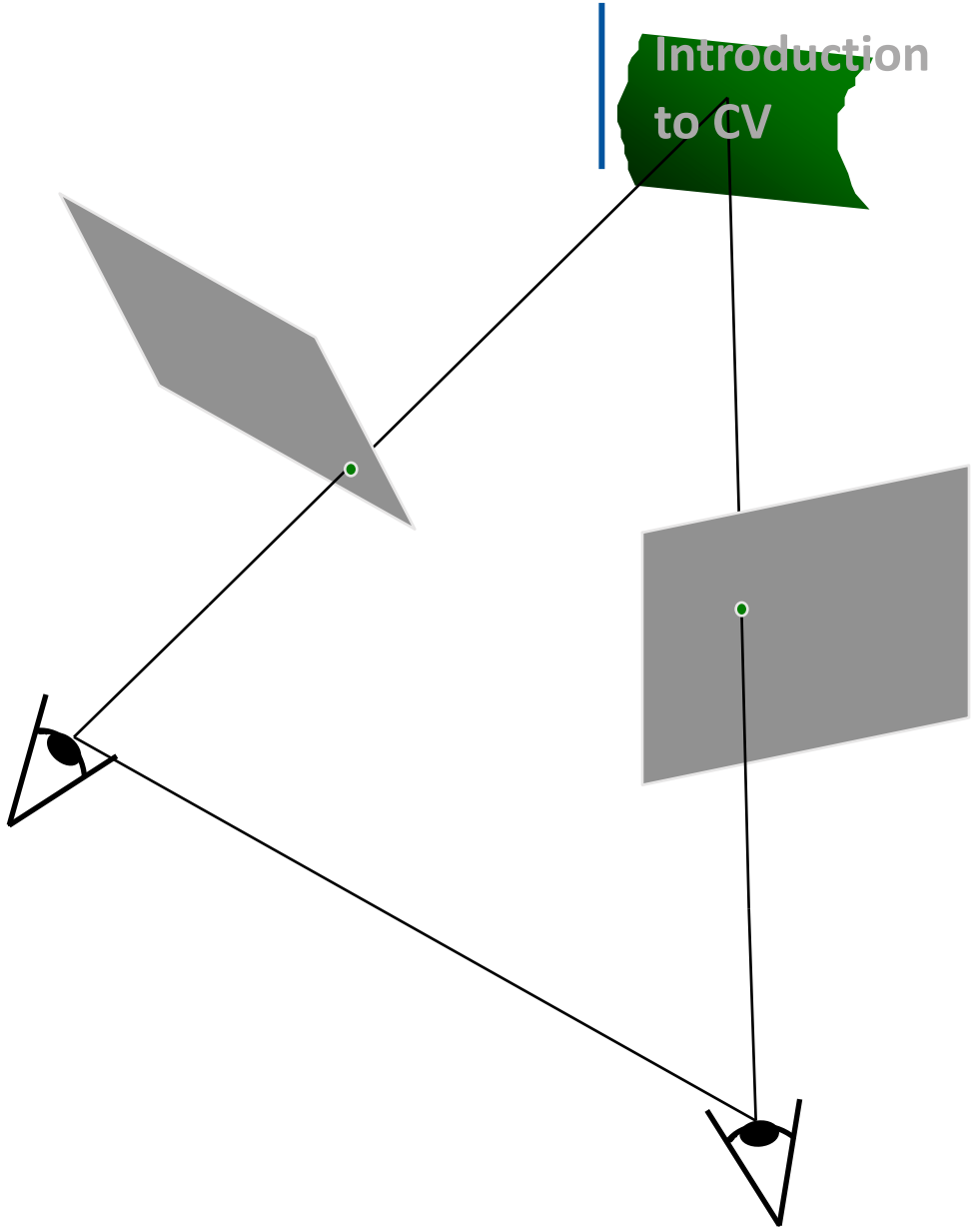
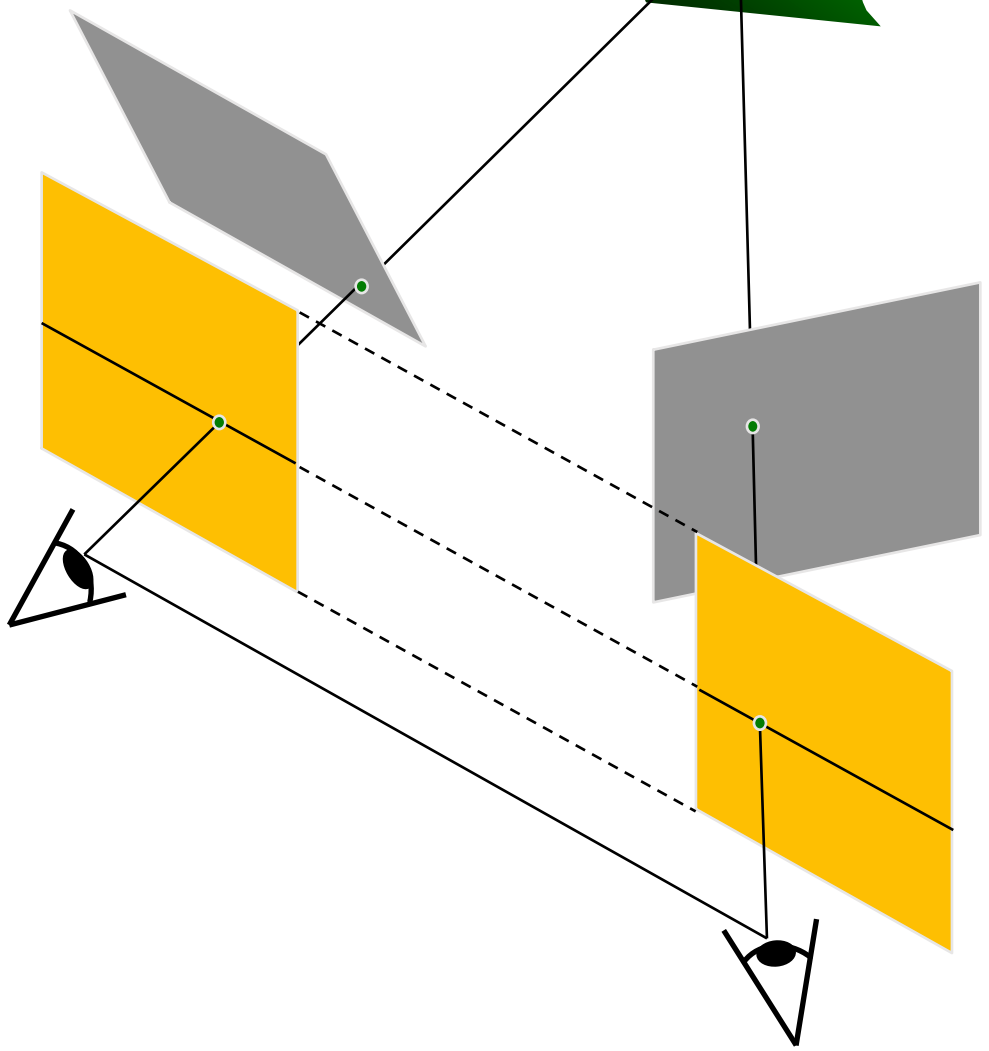


Fig. © S. Narasimhan, Computer Vision, CS, CMU, 2010

Stereo Image Rectification

Этапы ректификации. Плоскости изображений перепроецируются на общую плоскость, которая параллельна линии, соединяющей оптические центры камер. После такой трансформации смещение пикселей (диспаратность) становится **строго горизонтальным**.



Stages:

- reproject image planes onto a common plane parallel to the line between optical centers
- pixel motion is horizontal after this transformation

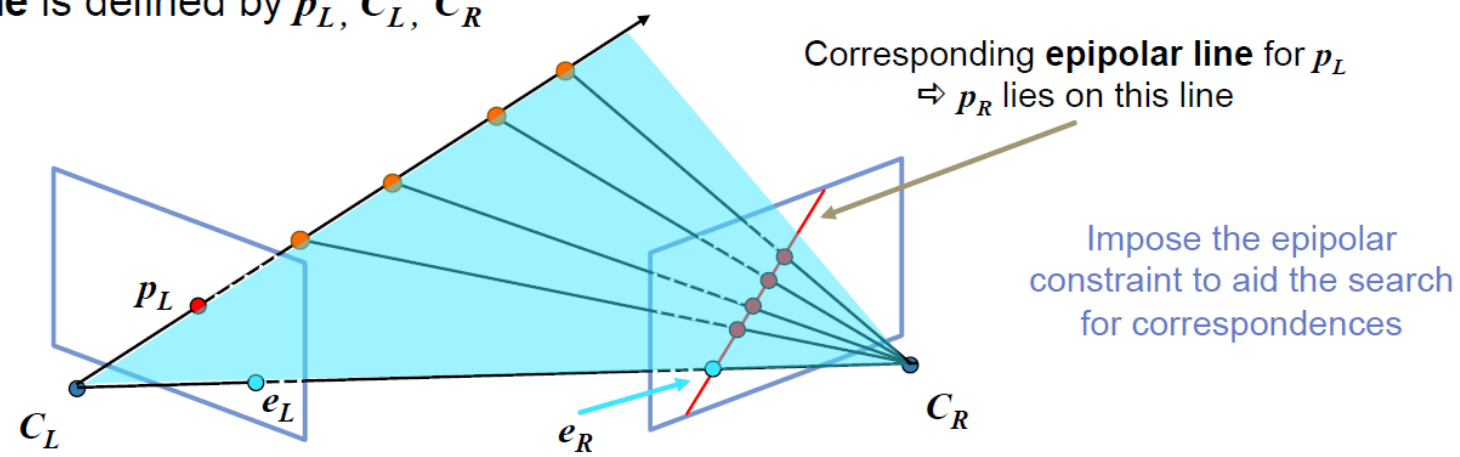
C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. IEEE Conf. Computer Vision and Pattern Recognition, 1999

Fig. © S. Narasimhan, Computer Vision, CS, CMU, 2010

Correspondence Search | the epipolar constraint

элементы эпиполярной геометрии. Вводится понятие **эпиполярной плоскости** (задается точкой объекта p_L и оптическими центрами (C_L, C_R), **эпиполярной линии** (проекция луча зрения на плоскость другой камеры) и **эпиполя**. Потенциальные совпадения для точки должны лежать строго на соответствующей эпиполярной линии.

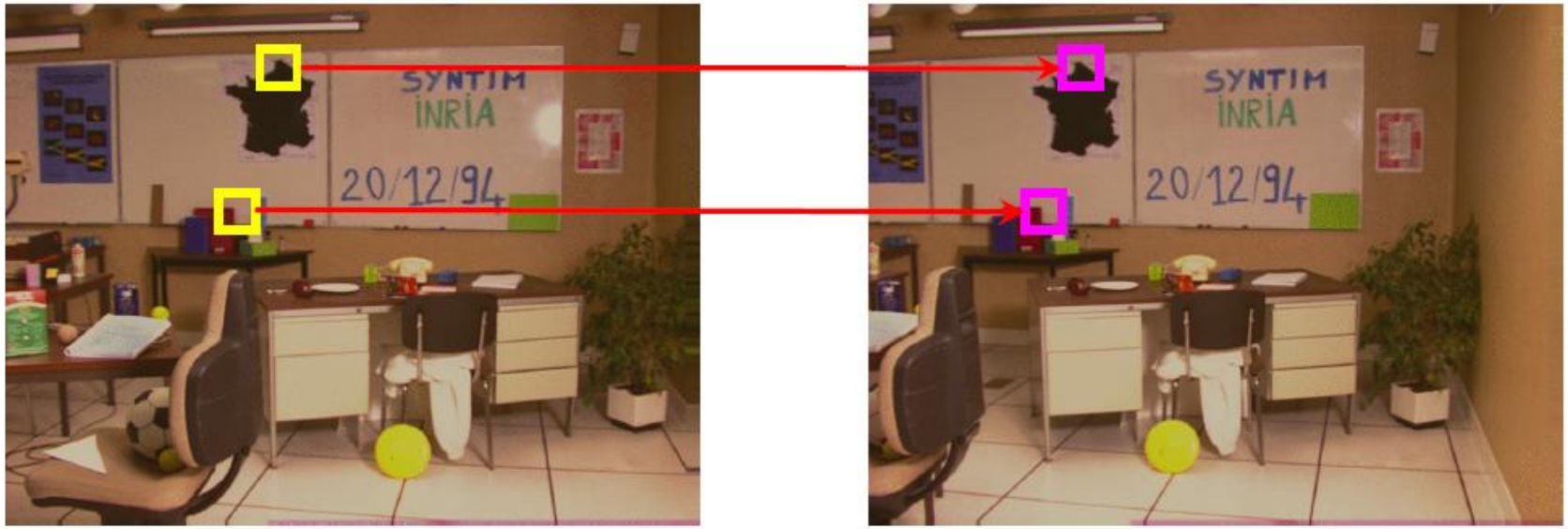
- Potential matches for p_L have to lie on the corresponding epipolar line
- **Epipolar line**: the projection of the infinite ray that connects $C_L - p_L$, onto the image plane of C_R (and vice versa)
- **Epipole**: the projection of the optical center C_L onto the image plane of C_R (and vice versa)
- **Epipolar plane** is defined by p_L, C_L, C_R



© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Correspondence Search | the problem

- **goal**: identify image regions / patches in the left & right images, corresponding to the same scene structure
  - Typical **similarity measures**: Normalized Cross-Correlation (NCC) , Sum of Squared Differences (SSD), Sum of Absolute Differences (SAD), …
  - **Exhaustive** image search can be computationally very expensive!
    Can we search for correspondences more efficiently?



© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Correspondence Search | the epipolar constraint

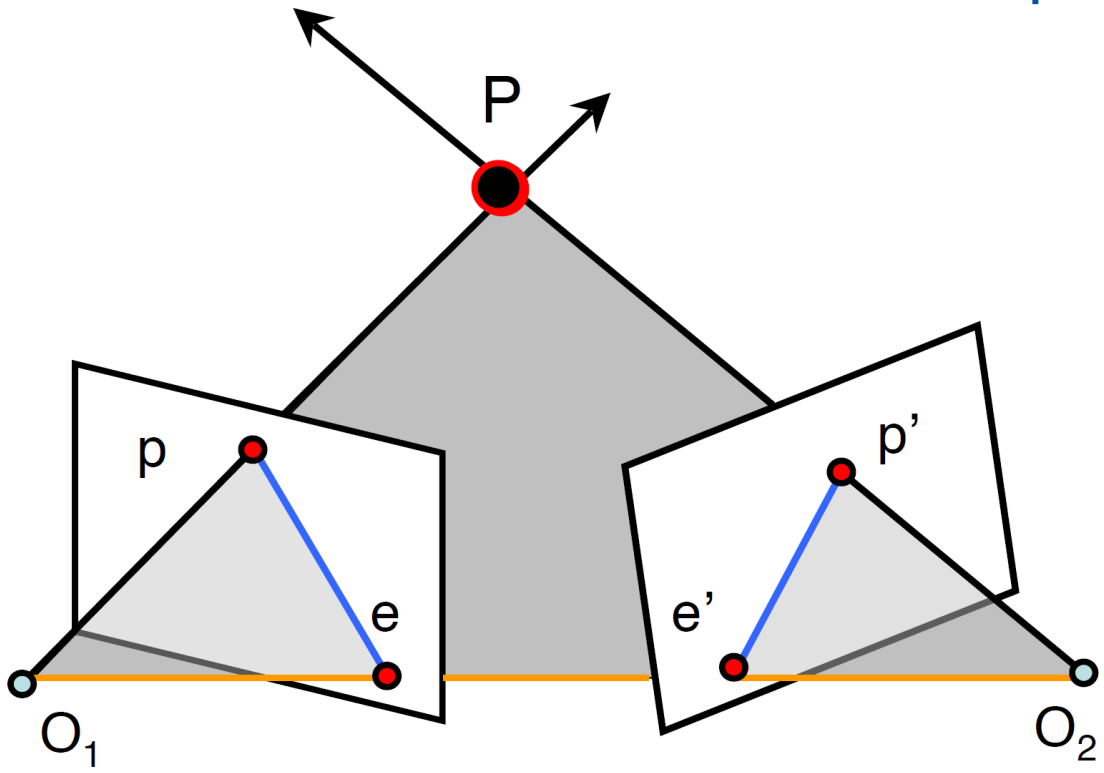
- Thanks to the epipolar constraint, corresponding points can be searched for, along epipolar lines
  ⇒ computational cost reduced to 1 dimension!



© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart,
Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Epipolar geometry

Обобщающая геометрическая схема (из классического курса Стэнфорда). Наглядно иллюстрируются взаимное расположение трехмерной точки P, её проекций *p, p'*, оптических центров O_1, O_2, базиса (Baseline), эпиполей *e, e'* и эпиполярных линий.



- Epipolar Plane
- Baseline
- Epipolar Lines

- Epipoles e, e'

  = intersections of baseline with image planes
  = projections of the other camera center

Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Example of epipolar lines

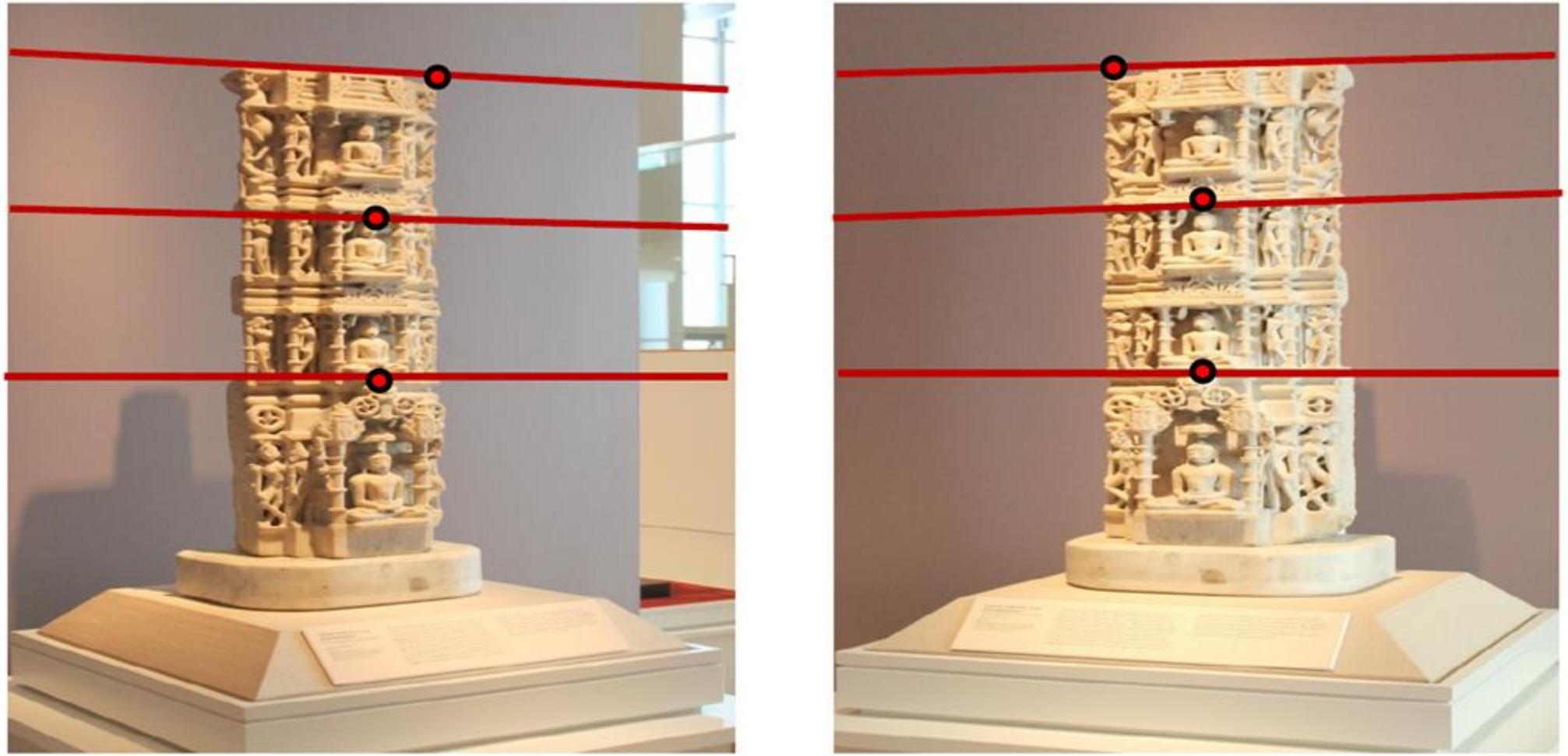
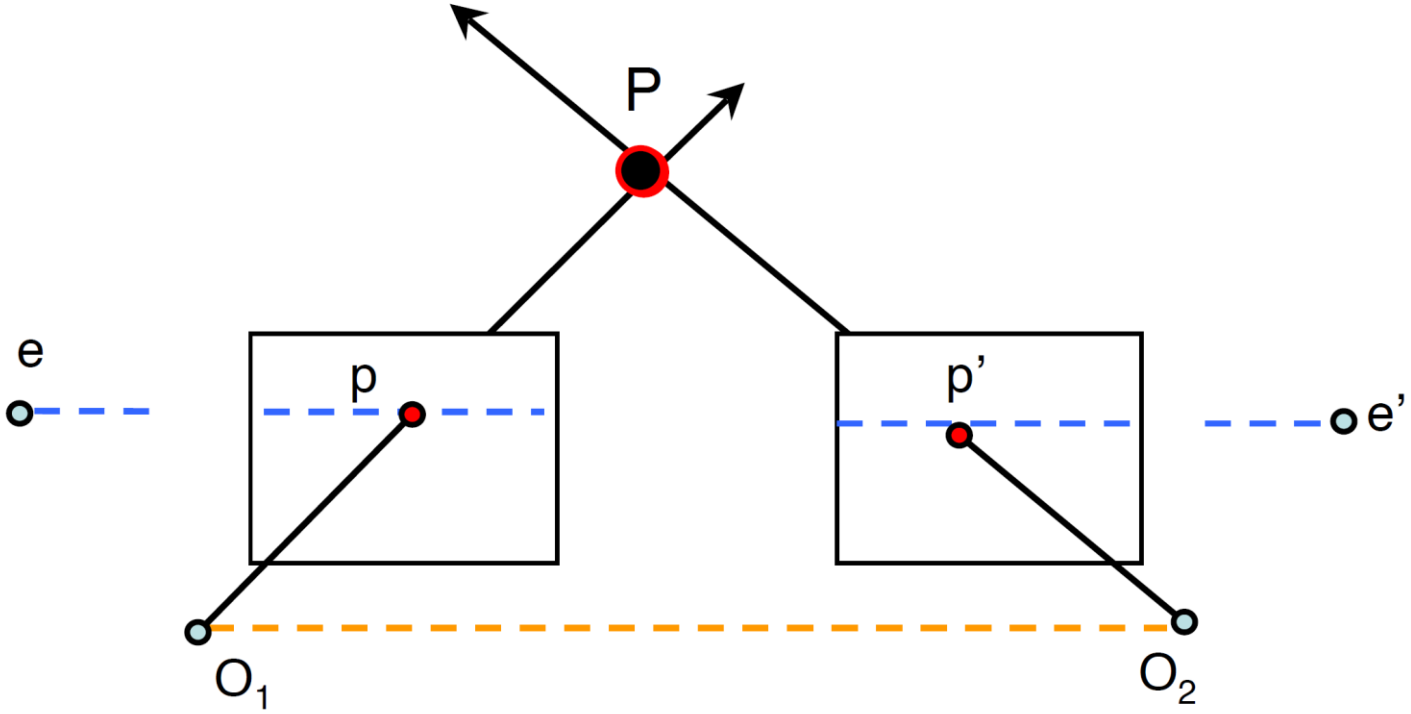


Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture5_epipolar_geometry_notes.pdf

Example: Parallel image planes

Геометрия идеального случая. Если плоскости изображений параллельны, базовая линия пересекает их в бесконечности. **Эпиполи уходят в бесконечность**, а эпиполярные линии становятся строго параллельны горизонтальной оси *X*.



- Baseline intersects the image plane at infinity
- Epipoles are at infinity
- Epipolar lines are parallel to x axis

Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Example: Parallel image planes

Пример параллельной конфигурации на реальных изображениях домов. Смещение пикселей (диспаратность) происходит строго по горизонтали, что позволяет искать соответствия исключительно вдоль одной строки.

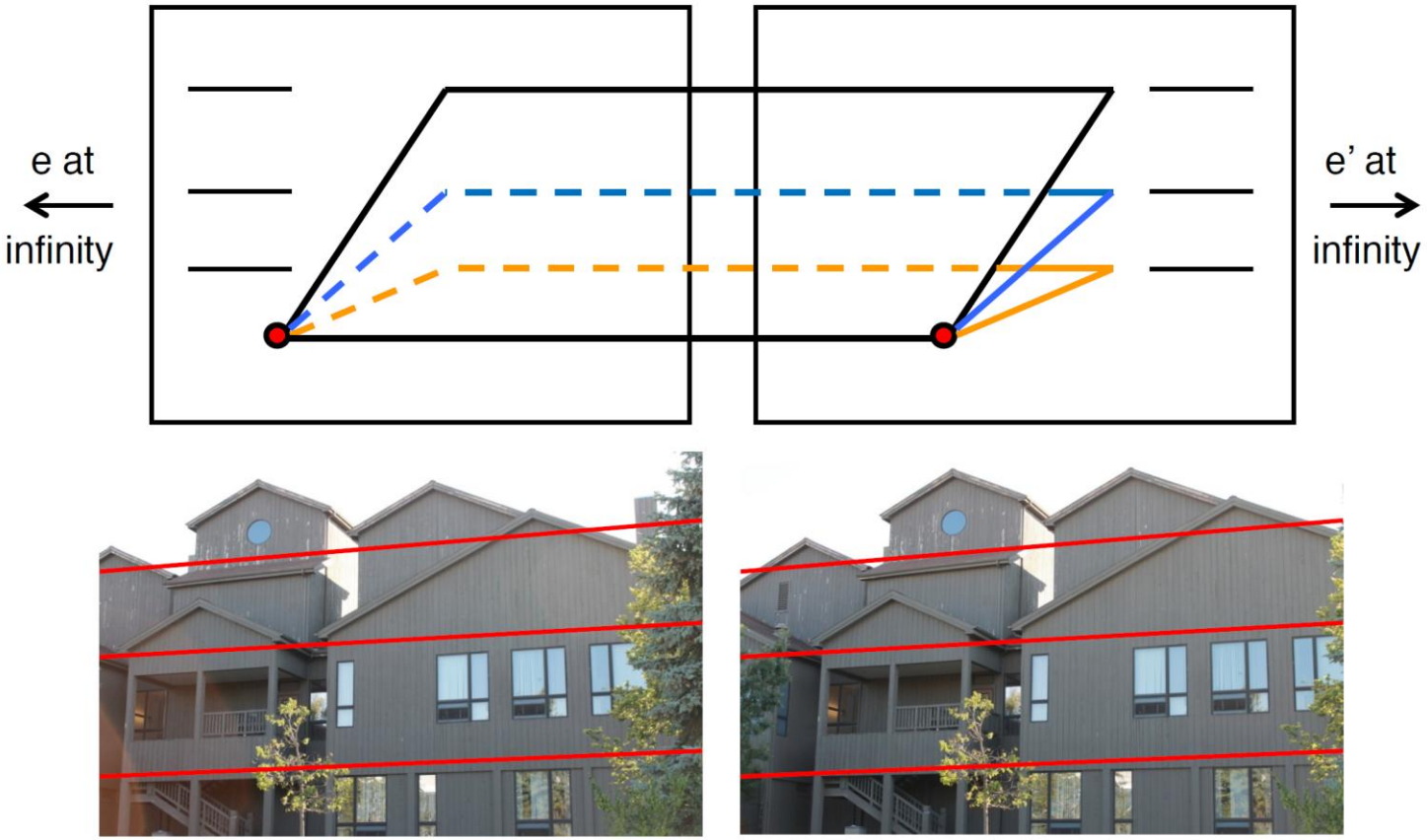
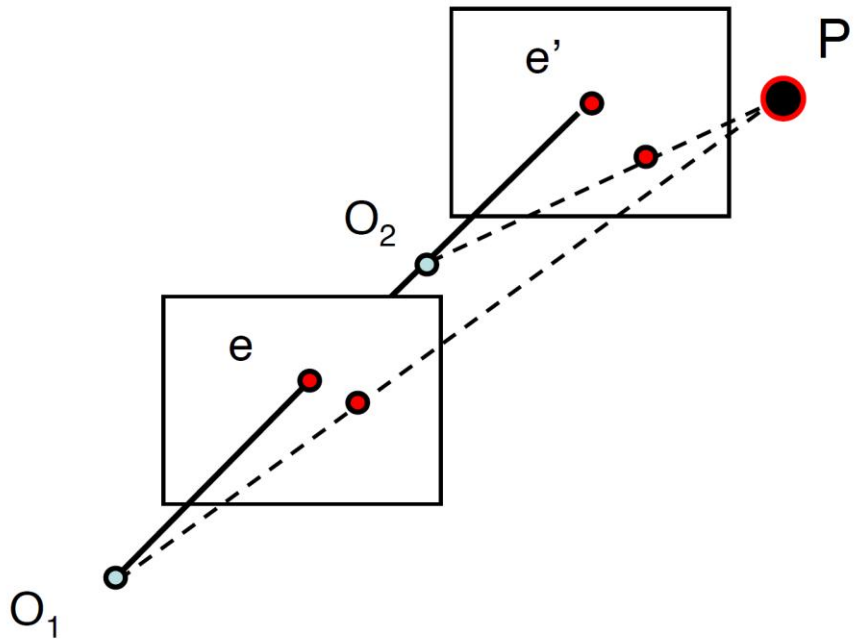


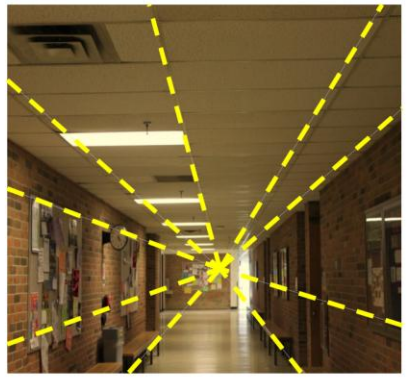
Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Example: Forward translation

Случай движения камеры строго вперед вдоль оптической оси. В такой конфигурации **эпиполи находятся внутри кадра** в одной и той же позиции для обоих изображений. Точка пересечения линий называется **FOE (Focus of Expansion / Фокус расширения)**, а эпиполярные линии радиально расходятся из этой центральной точки (напоминая вид из лобового стекла автомобиля в туннеле).



- The epipoles have same position in both images
- Epipole called FOE (focus of expansion)



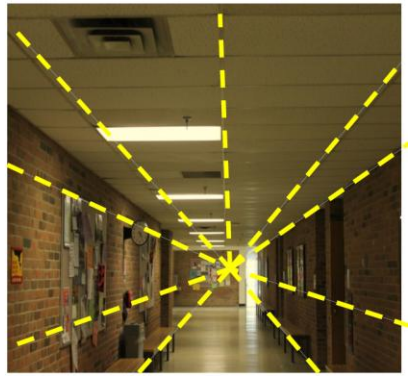


Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Epipolar Constraint



- Two views of the same object
- Suppose I know the camera positions and camera matrices
- Given a point on left image, how can I find the corresponding point on right image?

Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Epipolar geometry

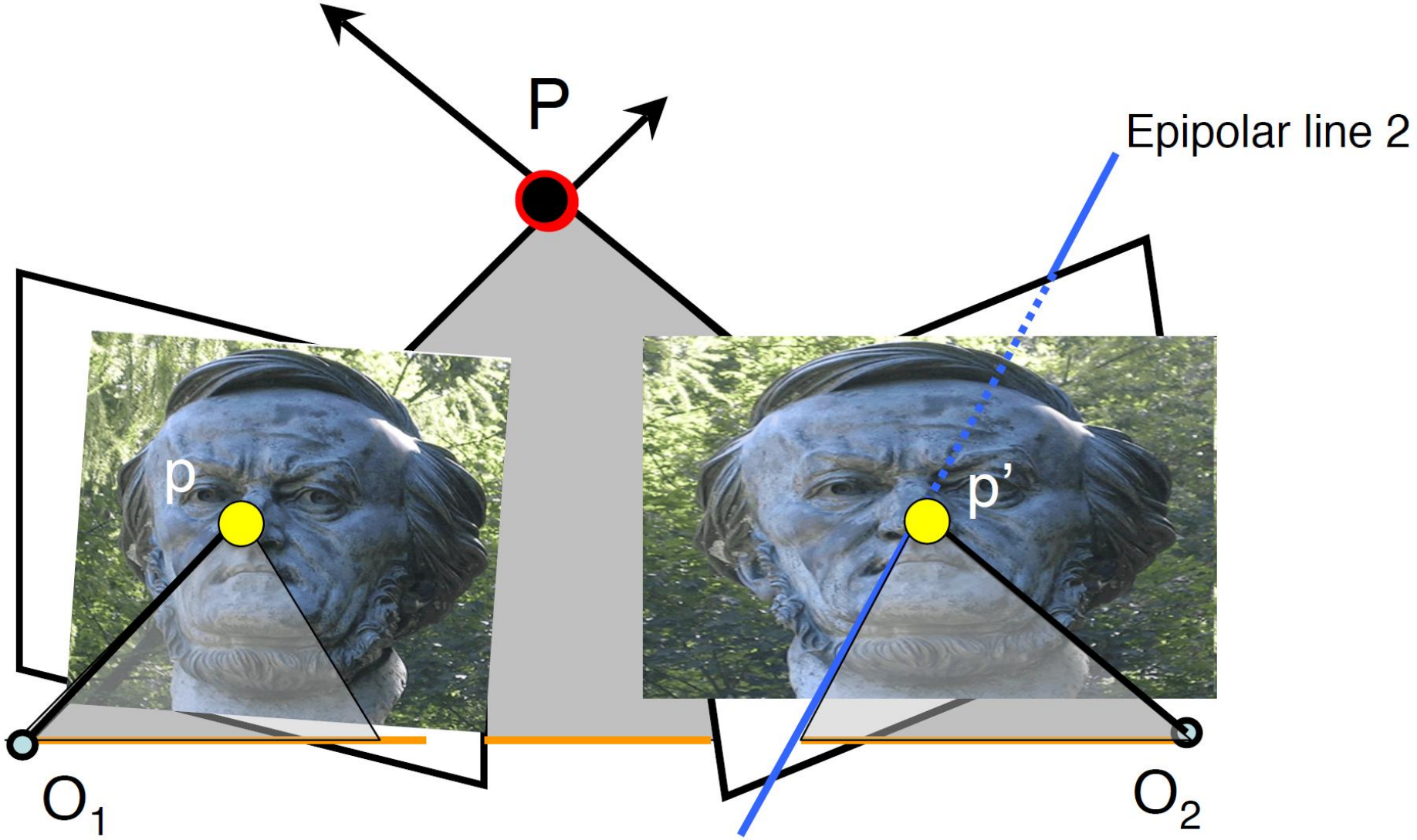


Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Epipolar Constraint

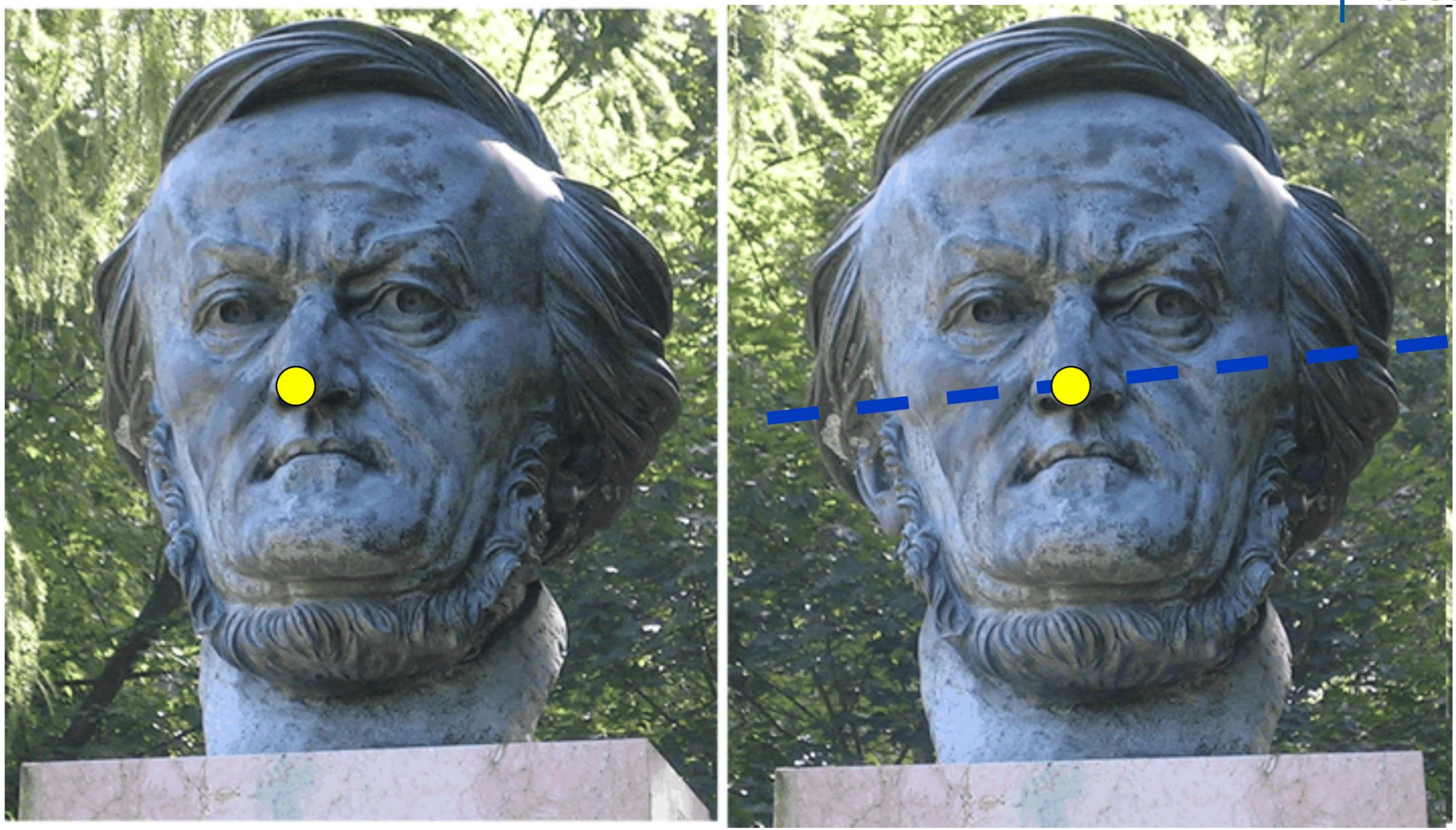


Fig. © "Epipolar geometry" by Prof. Silvio Savarese (Stanford University), 2015: http://cvgl.stanford.edu/teaching/cs231a_winter1415/lecture/lecture2_camera_models.pdf

Epipolar Rectification

- In practice, it is convenient if image scan-lines are the epipolar lines
- **Epipolar rectification:** warps image pairs into new “rectified” images, whose epipolar lines are **parallel** & **collinear** (aligned to the baseline)

Rotation

Image from Left Camera

Image from Right Camera



© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Epipolar Rectification

- In practice, it is convenient if image scan-lines are the epipolar lines
- **Epipolar rectification:** warps image pairs into new “rectified” images, whose epipolar lines are **parallel** & **collinear** (aligned to the baseline)

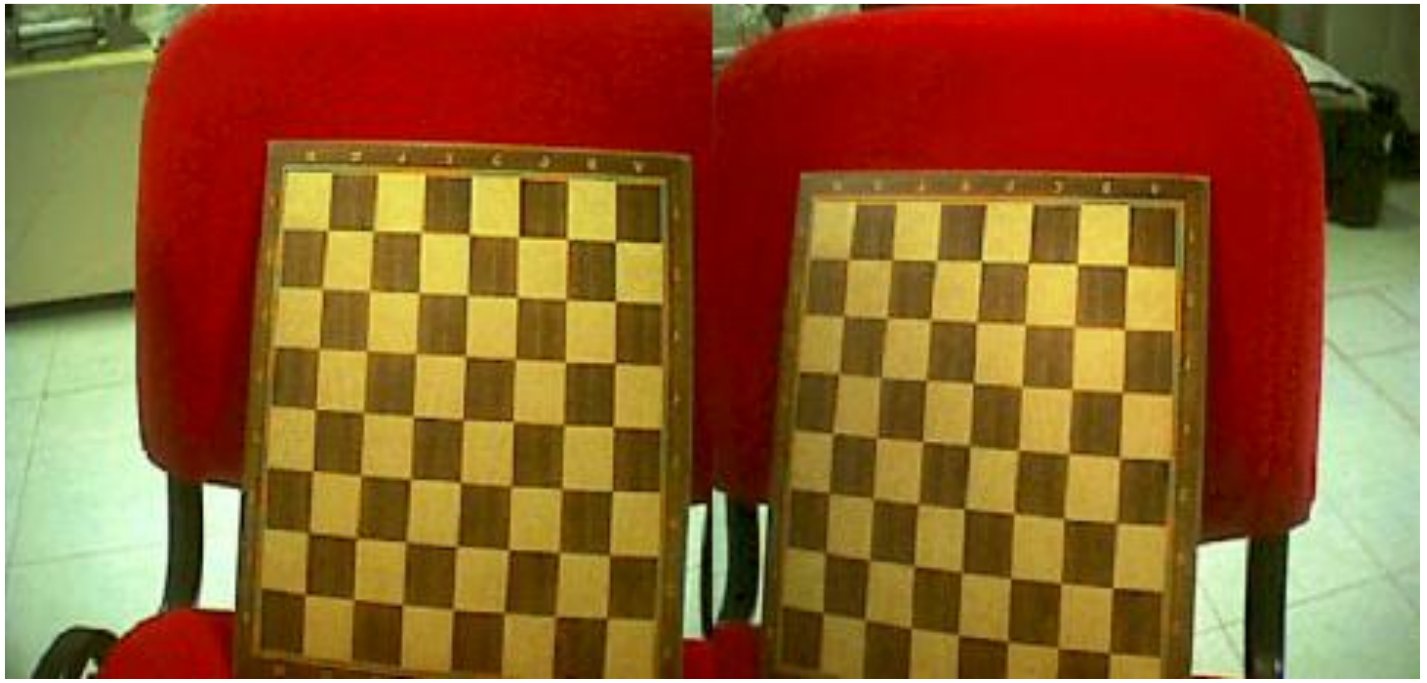
Rotation



Focal lengths

Image from Left Camera

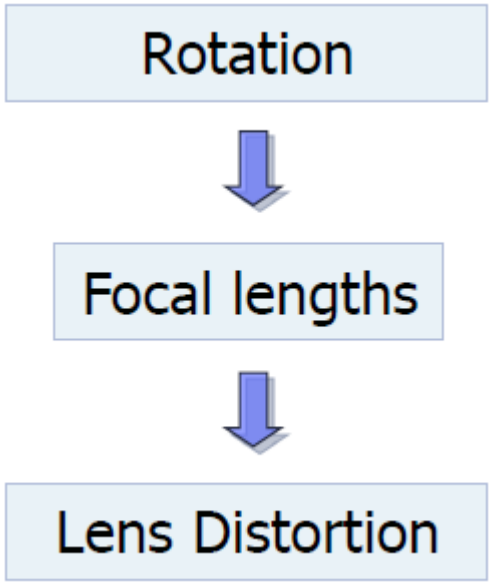
Image from Right Camera

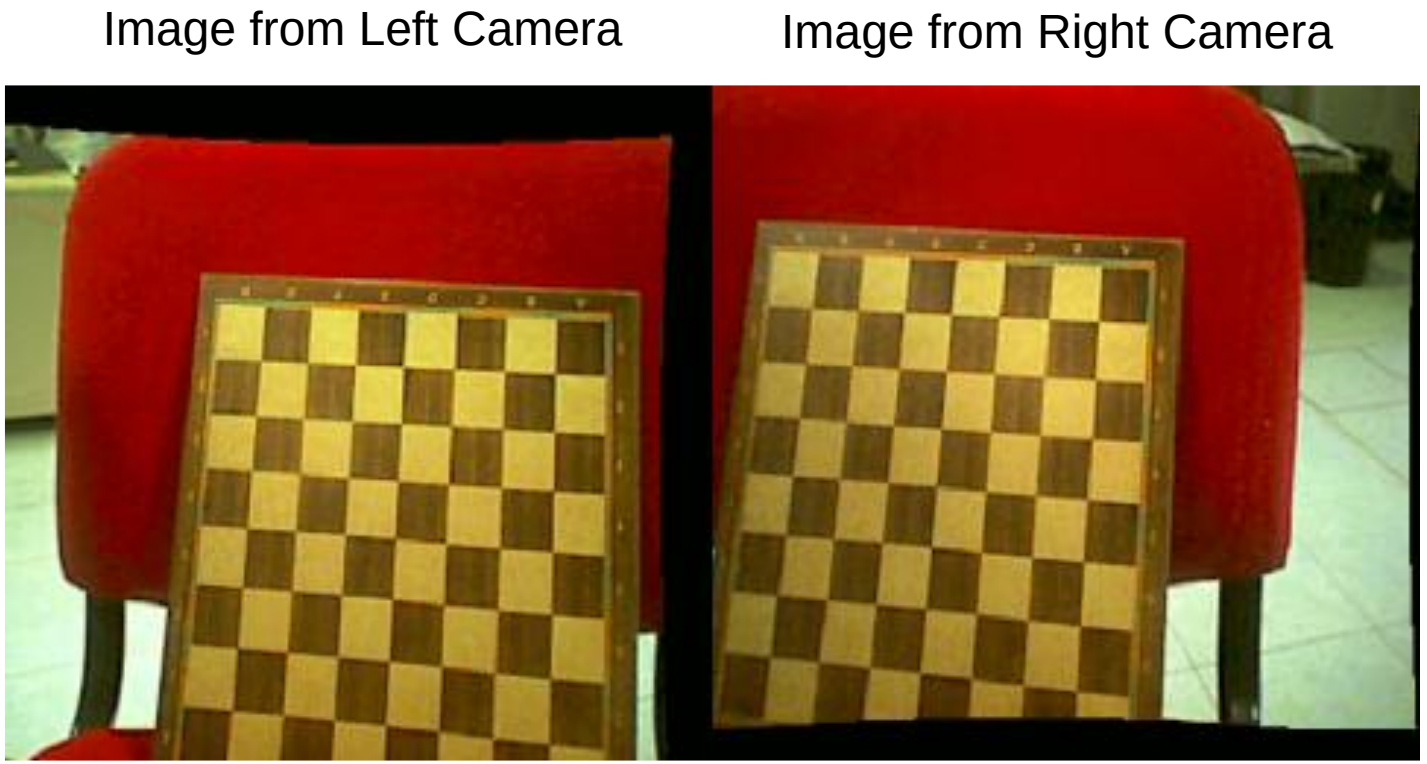


© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Epipolar Rectification

- In practice, it is convenient if image scan-lines are the epipolar lines
- **Epipolar rectification:** warps image pairs into new “rectified” images, whose epipolar lines are **parallel** & **collinear** (aligned to the baseline)

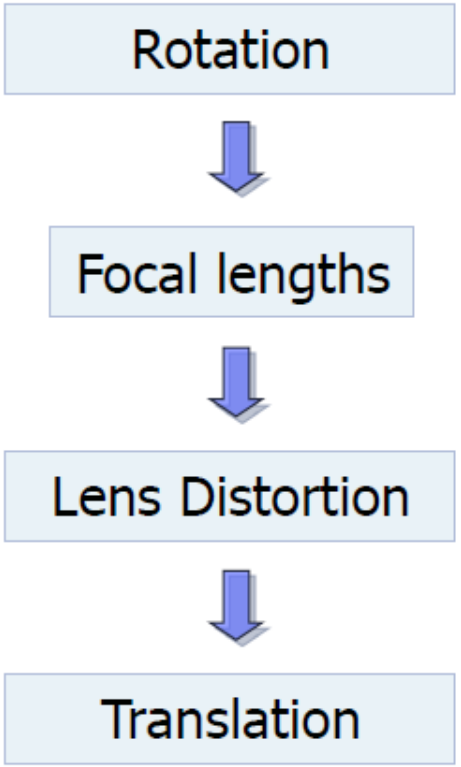


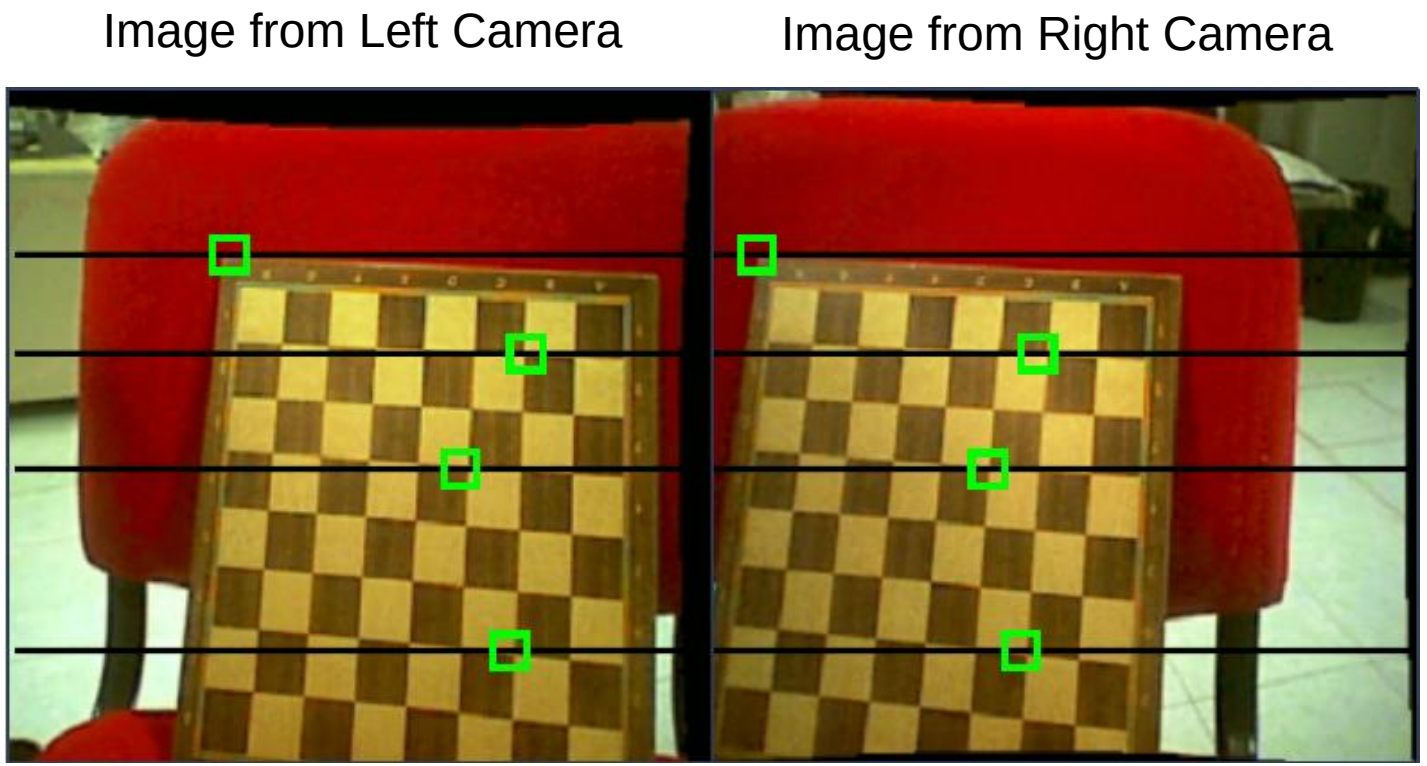


© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Epipolar Rectification

- In practice, it is convenient if image scan-lines are the epipolar lines
- **Epipolar rectification:** warps image pairs into new “rectified” images, whose epipolar lines are **parallel** & **collinear** (aligned to the baseline)





© Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Epipolar Rectification: example

Исходные ракурсы камер расположены под углами. После ректификации изображения трансформируются и выравниваются так, что поиск одинаковых точек сцены теперь происходит строго на одной горизонтальной линии.

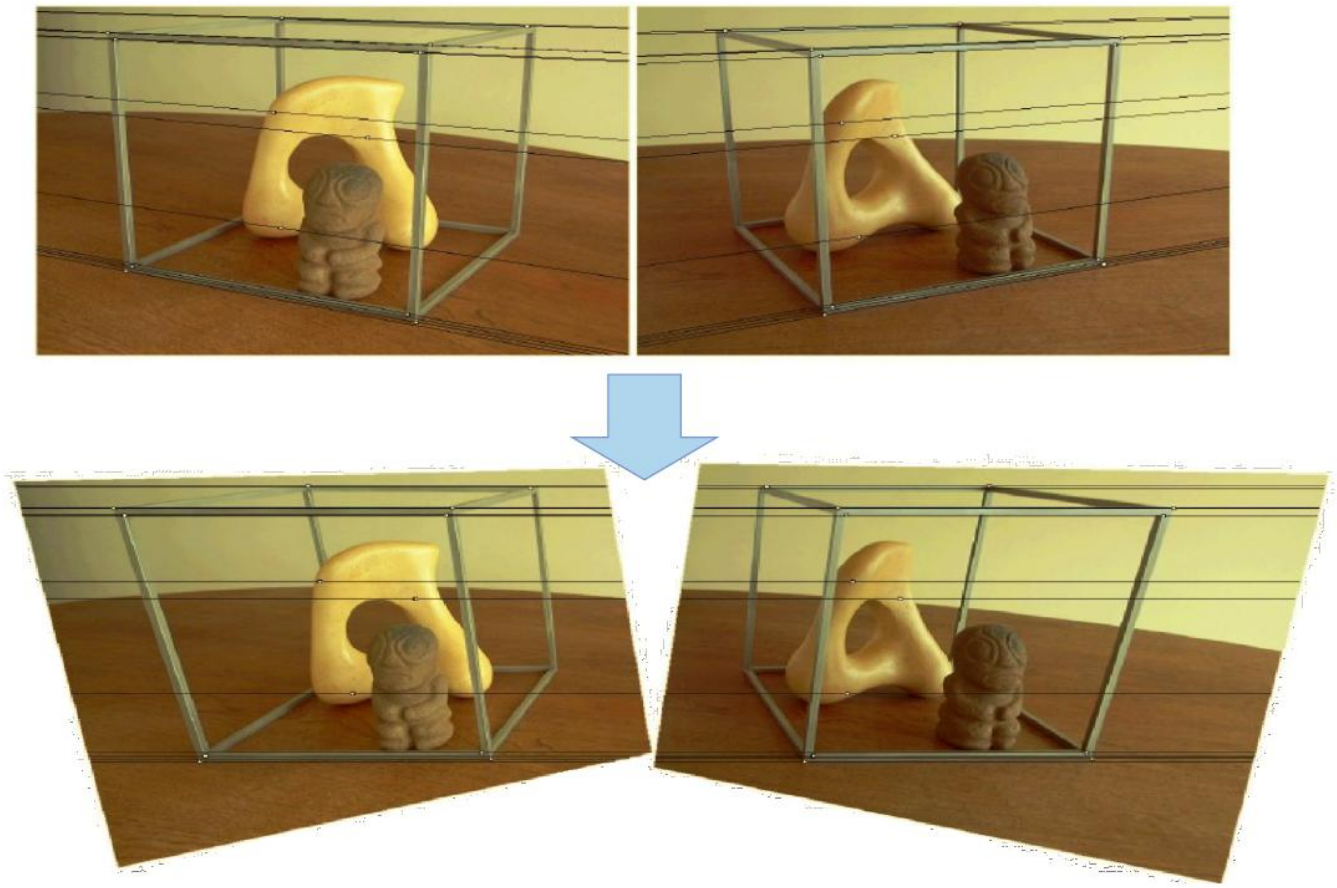


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

3D from stereo images

Постановка проблемы. Из-за шумов на изображениях и численных погрешностей лучи зрения из оптических центров C_1 и C_2 к точке P **не пересекаются в пространстве точно**. Задача — найти оптимальные координаты 3D-точки.

- Given the projections $\mathbf{p_1}$ and $\mathbf{p_2}$ of a 3D point $\mathbf{P}$ in two or more images (with known camera matrices $\boldsymbol{R}$ and $\boldsymbol{T}$), find the coordinates of the 3D point
- We want to intersect the two visual rays corresponding to $\mathbf{p_1}$ and $\mathbf{p_2}$, but because of noise and numerical errors, they don't meet exactly

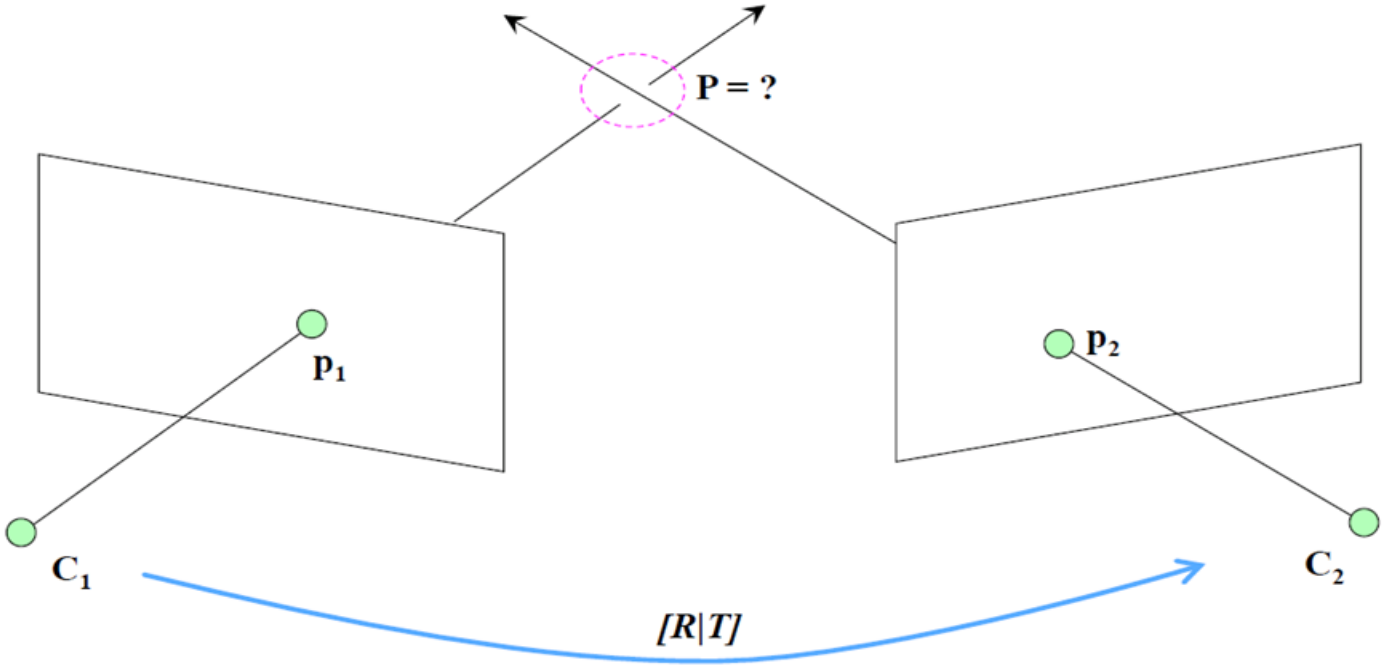


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Triangulation | geometric approach

Геометрический метод решения. Находится **кратчайший отрезок**, соединяющий два непересекающихся луча зрения, а искомая 3D-точка *P* определяется как его середина.

- Find shortest segment connecting the two viewing rays and let P be the midpoint of that segment

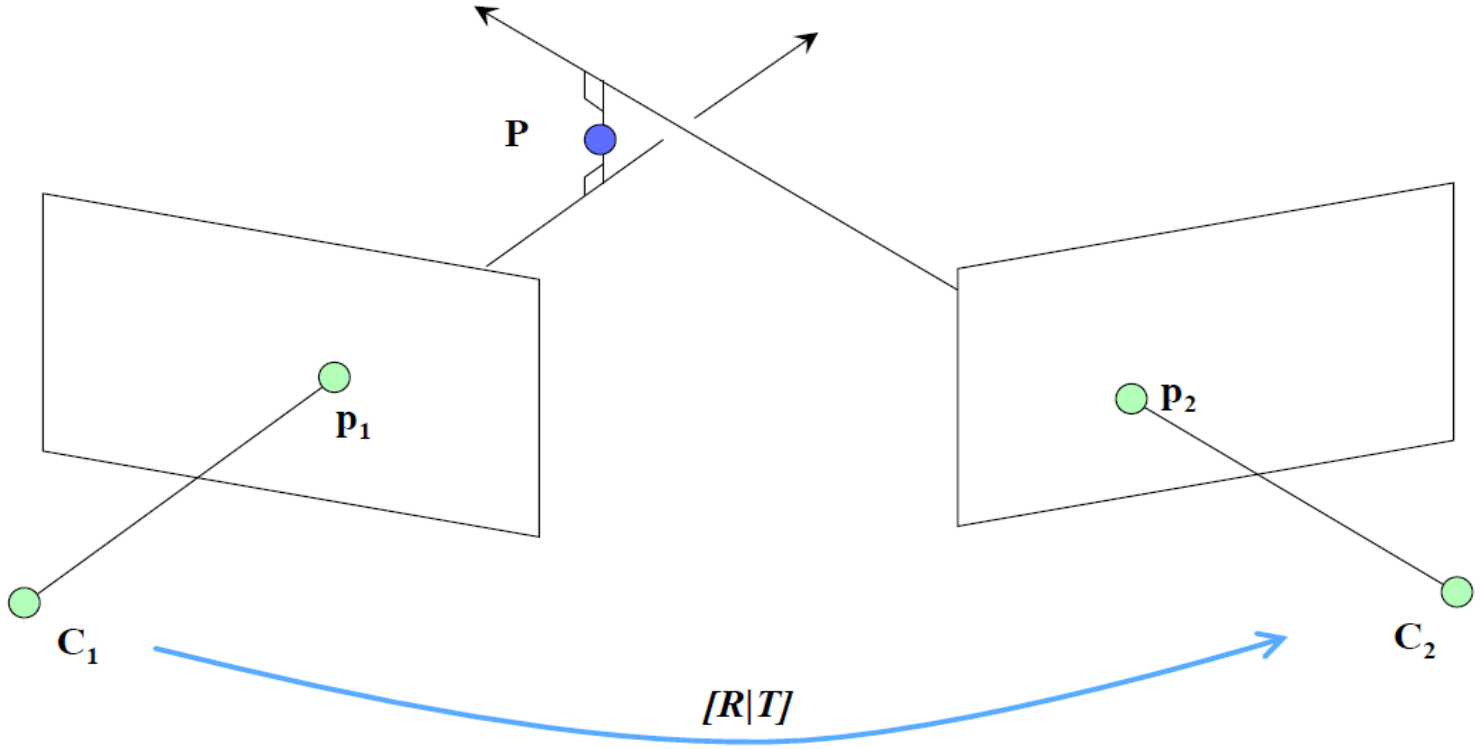
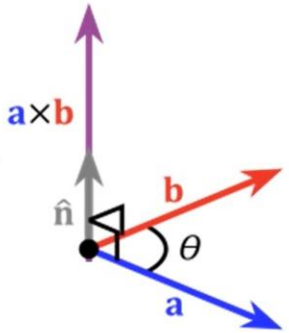


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Cross Product (or Vector Product) | review

Математический базис. Свойства векторного произведения $\left(\vec{a} \times \vec{b}\right)$ с равенством нулю для параллельных векторов. Векторное произведение представляется в виде умножения **кососимметричной матрицы** $[\vec{a}]_\times$ на вектор $\vec{b}$.

$$\vec{a} \times \vec{b} = \vec{c}, \quad \|\vec{c}\| = \|\vec{a}\|\|\vec{b}\|\sin(\theta) \cdot \vec{n}$$



- Vector cross product takes two vectors and returns a third vector that is perpendicular to both inputs

$$\vec{a} \cdot \vec{c} = 0$$

$$\vec{b} \cdot \vec{c} = 0$$

- The cross product of two parallel vectors = 0
- The vector cross product also can be expressed as the product of a skew-symmetric matrix and a vector

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} = [\mathbf{a}_\times]\mathbf{b}$$

Triangulation | linear approach

Алгебраический метод (линейный подход). Условие параллельности луча зрения и вектора на 3D-точку для левой и правой камер записывается через векторное произведение: $p \times M P = 0$. С использованием кососимметричных матриц $[p_\times] M P = 0$ это преобразуется в однородную систему линейных уравнений для нахождения координат точки *P*.

Left camera:

$$\lambda_2 \begin{bmatrix} u_2 \\ v_2 \\ 1 \end{bmatrix} = K_1 \begin{bmatrix} I | 0 \end{bmatrix} \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \Rightarrow \lambda_1 p_1 = M_1 \cdot P \Rightarrow p_1 \times M_1 \cdot P = 0 \Rightarrow \left[p_{1\times} \right] M_1 \cdot P = 0$$

Right Camera:

$$\lambda_2 \begin{bmatrix} u_2 \\ v_2 \\ 1 \end{bmatrix} = K_2 \begin{bmatrix} R | T \end{bmatrix} \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \Rightarrow \lambda_2 p_2 = M_2 \cdot P \Rightarrow p_2 \times M_2 \cdot P = 0 \Rightarrow \left[p_{2\times} \right] M_2 \cdot P = 0$$

Cross product as matrix multiplication: $\mathbf{a} \times \mathbf{b} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} = [\mathbf{a}_\times]\mathbf{b}$

Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Triangulation | linear approach

Сформированная однородная система из 6 уравнений с 3 неизвестными (X_w, Y_w, Z_w) решается методом сингулярного разложения матрицы (**SVD**).

Left camera:

$$\lambda_2 \begin{bmatrix} u_2 \\ v_2 \\ 1 \end{bmatrix} = K_1 \left[I | 0 \right] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \Rightarrow \lambda_1 p_1 = M_1 \cdot P \Rightarrow p_1 \times M_1 \cdot P = 0 \Rightarrow \left[p_{1\times} \right] M_1 \cdot P = 0$$

Right Camera:

$$\lambda_2 \begin{bmatrix} u_2 \\ v_2 \\ 1 \end{bmatrix} = K_2 \left[R | T \right] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \Rightarrow \lambda_2 p_2 = M_2 \cdot P \Rightarrow p_2 \times M_2 \cdot P = 0 \Rightarrow \left[p_{2\times} \right] M_2 \cdot P = 0$$

3 unknowns (P) : X_w , Y_w , Z_w

6 independent equations

⇒ Solve system: $A \cdot P = 0$ using SVD

Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Triangulation | non-linear approach

Нелинейный подход. Описывается метод, минимизирующий **сумму квадратов ошибок репроекции (SSRE)** — расстояний на плоскостях изображений между реально наблюдаемыми точками (p_1, p_2) и проекциями вычисленной 3D-точки (*MP*). Для оптимизации используются итерационные методы Гаусса-Ньютона или Левенберга-Марквардта. Линейный метод со слайда 37 служит здесь начальным приближением.

- Find $\boldsymbol{P}$ that minimizes the Sum of Squared Reprojection Error:

$$SSRE = \|p_1 - M_1 P\|^2 + \|p_2 - M_2 P\|^2$$

- In practice: initialize $\boldsymbol{P}$ using linear approach and then minimize SSRE using Gauss-Newton or Levenberg-Marquardt

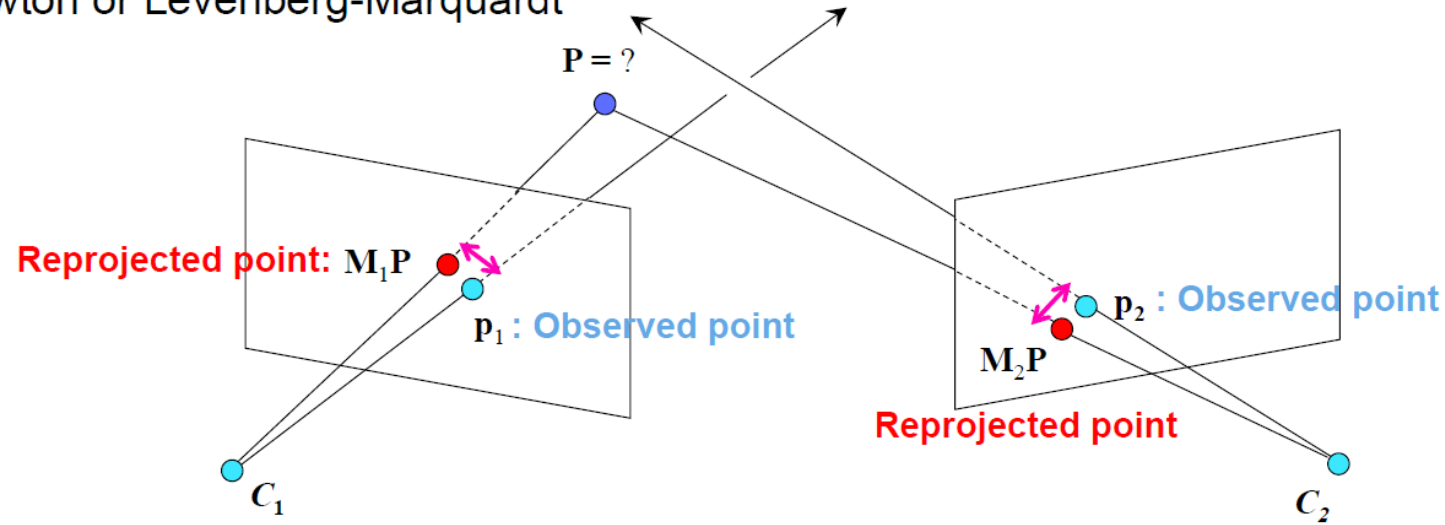
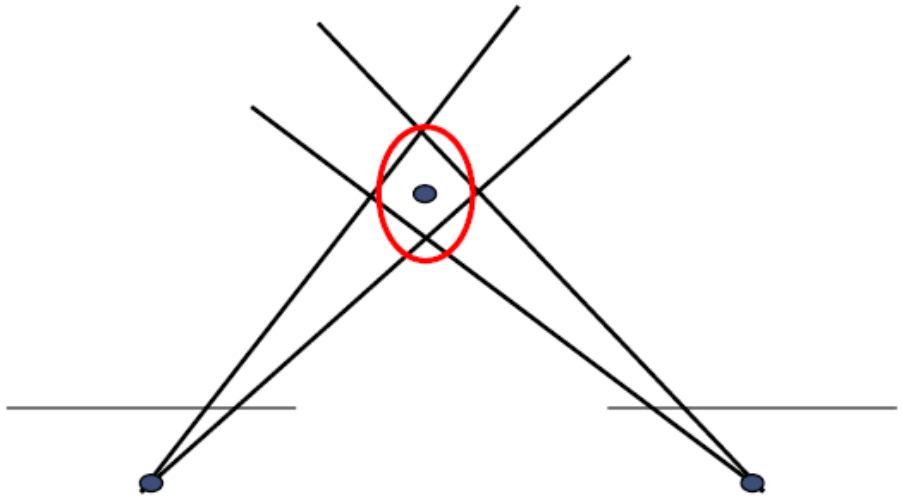


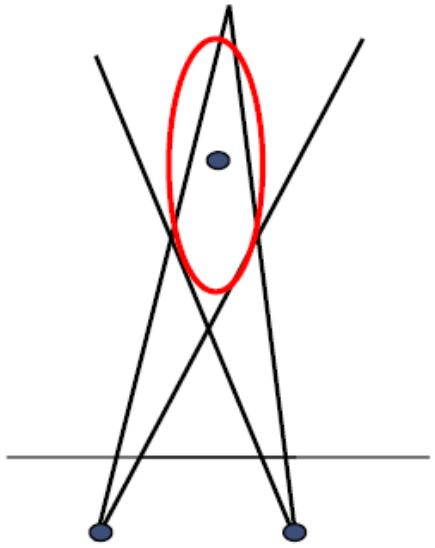
Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Triangulation | sensitivity to baseline

- What's the optimal baseline?
  - **Too small:**
    - Large depth error
    - Can you quantify the error as a function of the disparity?
  - **Too large:**
    - Object might be visible from one camera, but not the other
    - Difficult search problem for close objects



Large Baseline



Small Baseline

Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2

Калибровка стереокамеры

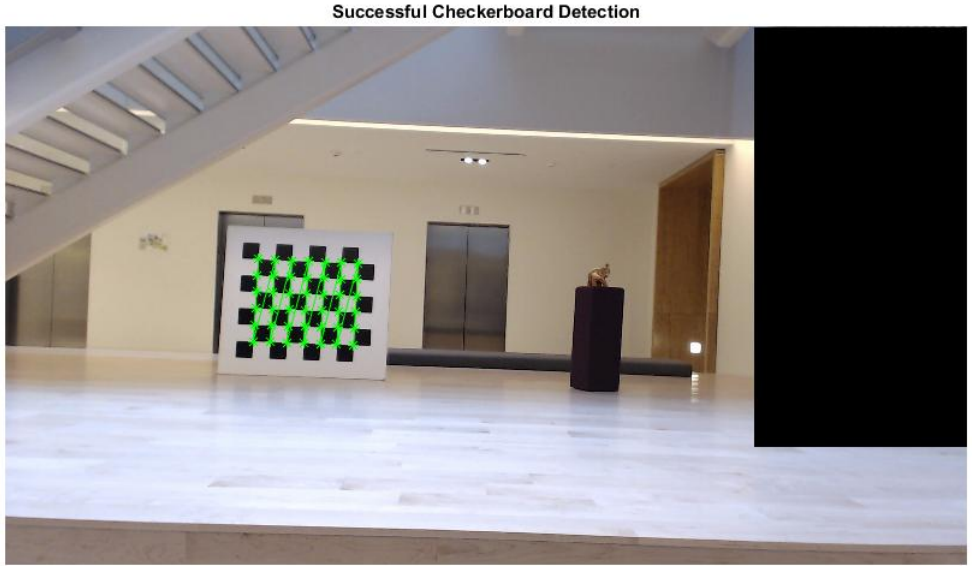
Термины: Калибровка стереокамер · Ошибка репроекции · Калибровочный шаблон (Шахматная доска) · Эпиполярная ректификация (Stereo rectification) · Анаглифное изображение

Stereo Calibration and Scene Reconstruction

The process consists of the following steps:

- Calibrate the stereo camera system
- Rectify a pair of stereo images
- Compute disparity
- Reconstruct the 3-D point cloud

Процесс калибровки с использованием калибровочного шаблона (шахматной доски) и график величины ошибки репроекции для разных кадров.



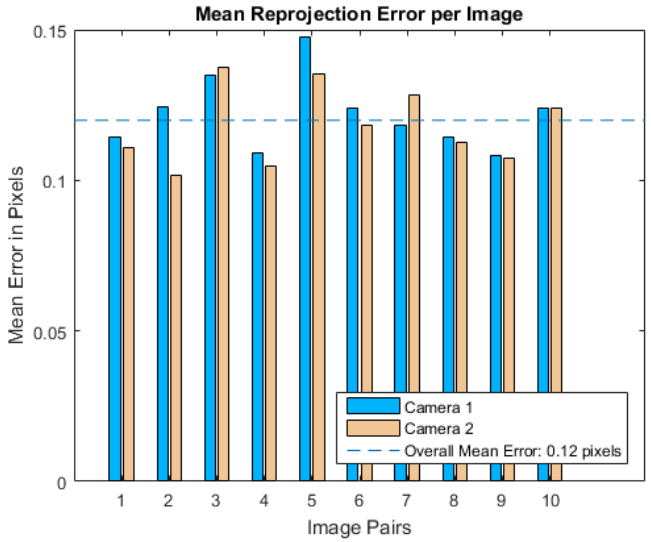


Fig. © MathWorks. Stereo Calibration and Scene Reconstruction

Calibration of a Stereo camera system

Практика калибровки. Процесс включает оценку внутренних параметров (intrinsics) каждой камеры, а также вектора переноса и матрицы поворота (extrinsics) второй камеры относительно первой. Требования к съемке: использовать асимметричную доску под разными углами на рабочем расстоянии и сохранять фото без потерь (в PNG).

- Calibration of a stereo camera system involves the estimation of the **intrinsic parameters of each camer**a. It also includes the estimation of the **translation and rotation** of the **second camera relative to the first** one.
- To calibrate the stereo system:
  - **use an asymmetric chessboard** as a calibration pattern (place the pattern at the same distance from the cameras as the objects you want to measure);
  - **take multiple pairs of images** of the chessboard **from different angles** (entire pattern must be visible in each image);
  - store the images in a format that does **not use lossy compression** (e.g. PNG).

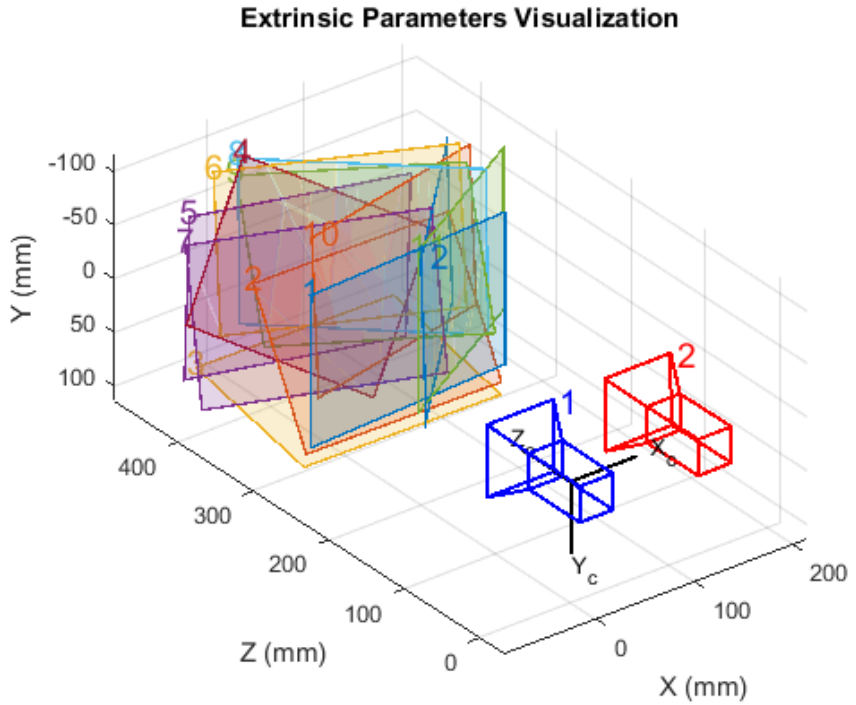


Fig. © MathWorks. Stereo Calibration and Scene Reconstruction

Stereo Calibration and Scene Reconstruction

The process consists of the following steps:

- Calibrate the stereo camera system
- Rectify a pair of stereo images
- Compute disparity
- Reconstruct the 3-D point cloud

Визуальный результат: анаглифные изображения стереопары до ректификации (со смещением по вертикали и углам) и после неё (с выровненными горизонтальными строками).

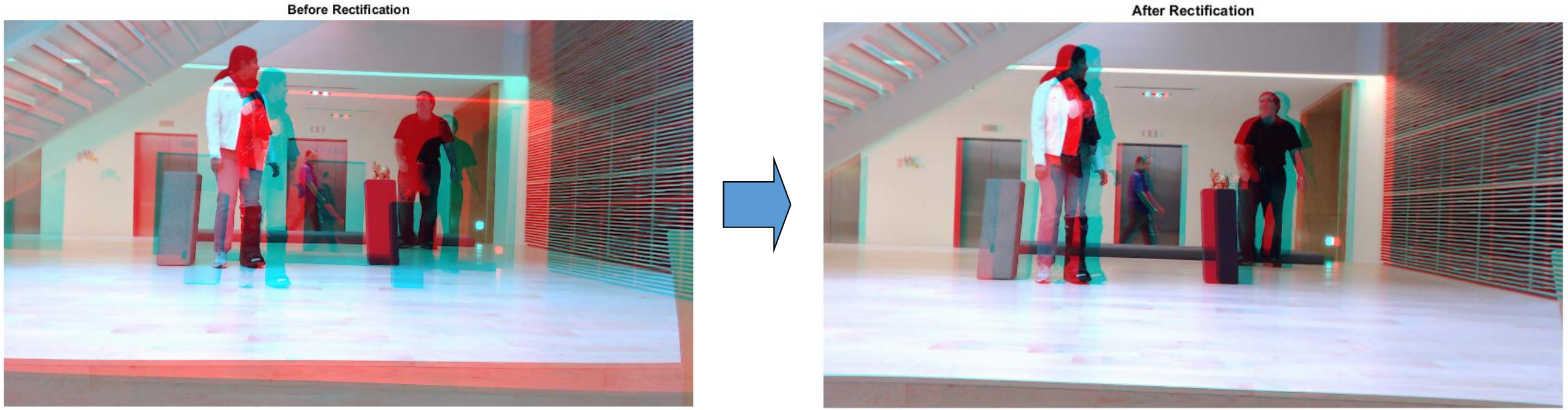


Fig. © MathWorks. Stereo Calibration and Scene Reconstruction

Stereo Calibration and Scene Reconstruction

The process consists of the following steps:

- Calibrate the stereo camera system
- Rectify a pair of stereo images
- Compute disparity
- Reconstruct the 3-D point cloud

На основе отректифицированных кадров рассчитывается плотная карта диспаратности (Disparity Map), где глубина закодирована цветом, что является основой для финального построения трехмерного облака точек.

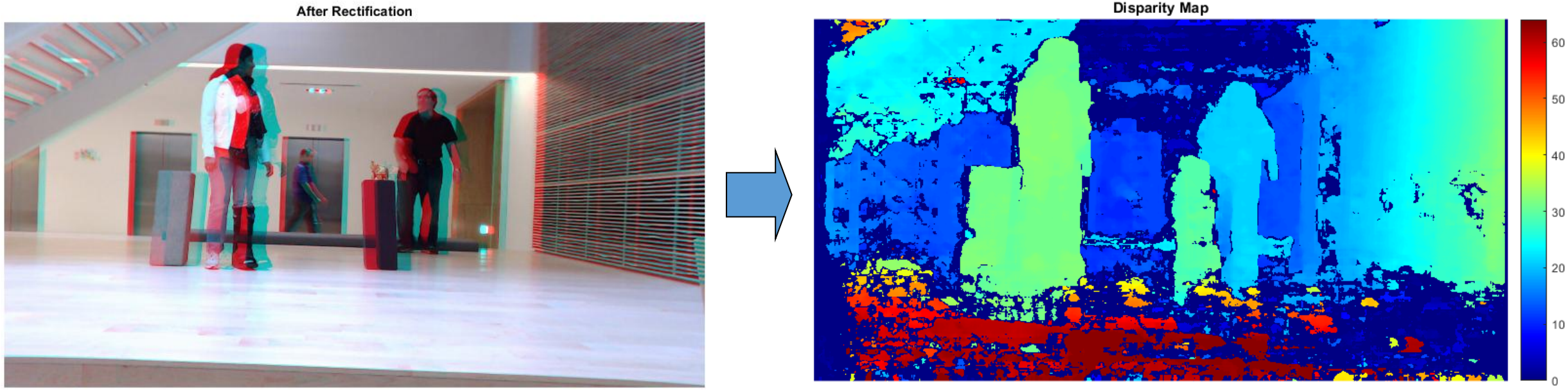


Fig. © MathWorks. Stereo Calibration and Scene Reconstruction

Stereo Calibration and Scene Reconstruction

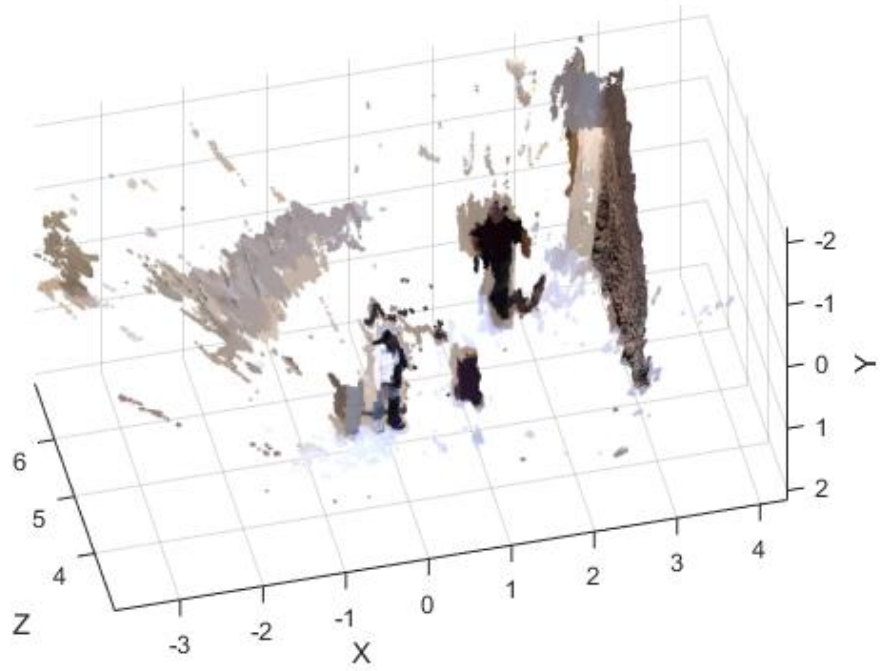
The process consists of the following steps:

- Calibrate the stereo camera system
- Rectify a pair of stereo images
- Compute disparity
- Reconstruct the 3-D point cloud



Fig. © MathWorks. Stereo Calibration and Scene Reconstruction

Переход от отректифицированной анаглифной картинки к полноценной **трехмерной реконструкции сцены (облаку точек)**, отображаемой в декартовой системе координат.



Stereo Example



left image | right image | depth map



Fig. © H. Tao et al. “Global matching criterion and color segmentation based stereo”

Importance of Stereo Calibration and Rectification

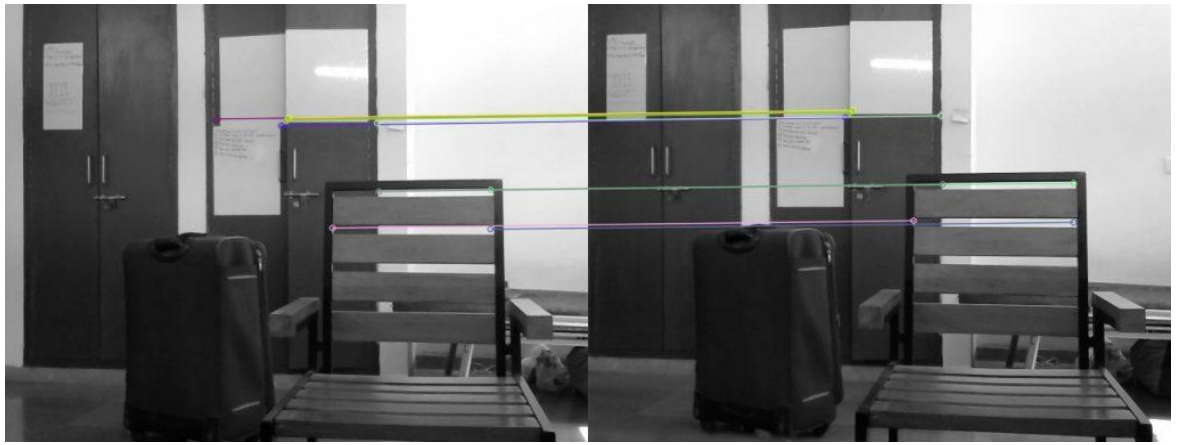


Если использовать «сырые» изображения с реальной стереопары, эпиполярные линии идут под углом, а карта диспаратности получается зашумленной и сильно искаженной.

To understand the importance of stereo calibration and stereo rectification, we try to generate a disparity map using the images captured from our stereo setup without any calibration or stereo rectification.



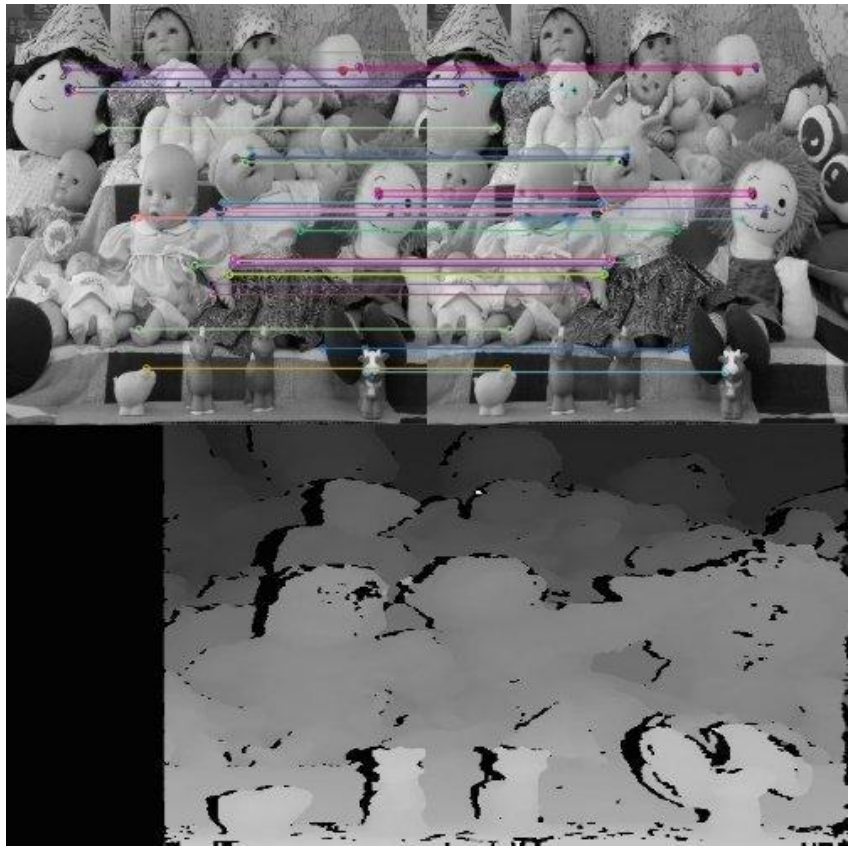
Disparity map generated using the right and left images without stereo calibration



When we plot the matching lines between a few corresponding points, we observe that the lines are not completely horizontal and the Y coordinates of corresponding points are not equal

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

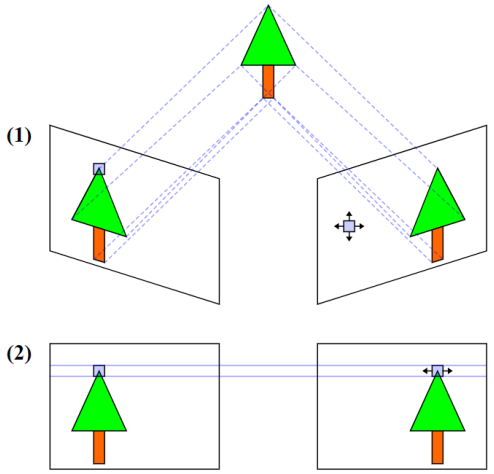
Importance of Stereo Calibration and Rectification



Feature matches for a stereo image pair and generated disparity map

- The Figure shows a pair of stereo images with point correspondence, and the disparity map generated using these images.
- We observe that now the disparity map is less noisy.
- In this case, the corresponding key points have equal Y coordinates. Such a case is only possible when the cameras are parallel. This is a special case of two-view geometry where the images are parallel and related by just a horizontal translation. This is essential because the method used to generate the disparity map searches only horizontal correspondence points.

- Instead of physically adjusting the cameras, we do this on the software side. We use a method called Stereo Image Rectification.
- The idea is to re-project the two images on a common plane parallel to the line passing through the optical centers.
- This ensures that the corresponding points have the same Y coordinate and are related simply by a horizontal translation.



Process of stereo rectification

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

Steps For Stereo Calibration and Rectification

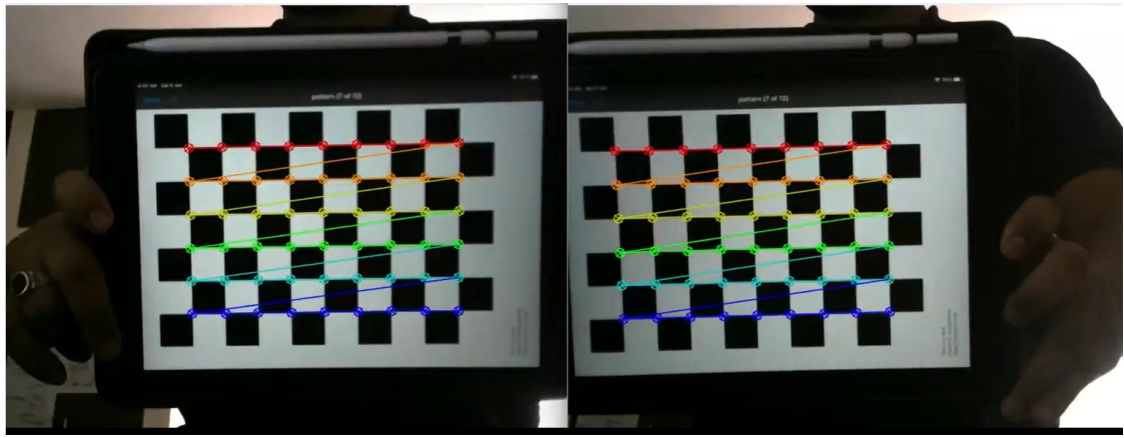
As we know, the images captured by a camera suffer from lens distortion. Hence apart from stereo rectification, it is also essential to undistort the images. So the overall process is as follows:

1. Calibrate individual cameras using the standard OpenCV calibration method explained in the camera calibration post.
2. Determine the transformation between the two cameras used in the stereo camera setup.
3. Using the parameters obtained in the previous steps and the stereoCalibrate method, we determine the transformations applied to both the images for stereo rectification.
4. Finally, a mapping required to find the undistorted and rectified stereo image pair is obtained using the initUndistortRectifyMap method.
5. This mapping is applied to the original images to get a rectified undistorted stereo image pair.

To perform these steps, we capture images of a calibration pattern

Стерео калибровка и ректификация:

- раздельная калибровка камер,
- определение взаимной трансформации между ними,
- вычисление параметров ректификации,
- создание карты искажений,
- финальное перепроецирование исходных изображений.



The process of capturing images for stereo calibration

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

Step 1: Individual calibration of the right and left cameras of the stereo setup


```python
# Set the path to the images captured by the left and right cameras
pathL = "./data/stereoL/"
pathR = "./data/stereoR/"

# Termination criteria for refining the detected corners
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)


objp = np.zeros((9*6,3), np.float32)
objp[:,:2] = np.mgrid[0:9,0:6].T.reshape(-1,2)

img_ptsL = []
img_ptsR = []
obj_pts = []

for i in tqdm(range(1,12)):
  imgL = cv2.imread(pathL+"img%d.png"%i)
  imgR = cv2.imread(pathR+"img%d.png"%i)
  imgL_gray = cv2.imread(pathL+"img%d.png"%i,0)
  imgR_gray = cv2.imread(pathR+"img%d.png"%i,0)

  outputL = imgL.copy()
  outputR = imgR.copy()

  retR, cornersR =  cv2.findChessboardCorners(outputR,(9,6),None)
  retL, cornersL = cv2.findChessboardCorners(outputL,(9,6),None)

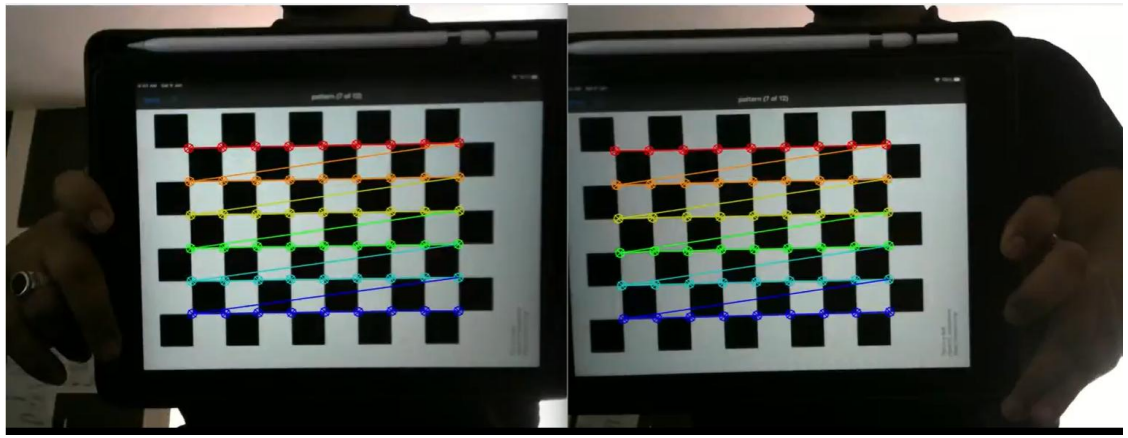
  if retR and retL:
    obj_pts.append(objp)
    cv2.cornerSubPix(imgR_gray,cornersR,(11,11),(-1,-1),criteria)
    cv2.cornerSubPix(imgL_gray,cornersL,(11,11),(-1,-1),criteria)
    cv2.drawChessboardCorners(outputR,(9,6),cornersR,retR)
    cv2.drawChessboardCorners(outputL,(9,6),cornersL,retL)
    cv2.imshow('cornersR',outputR)
    cv2.imshow('cornersL',outputL)
    cv2.waitKey(0)

    img_ptsL.append(cornersL)
    img_ptsR.append(cornersR)


# Calibrating left camera
retL, mtxL, distL, rvecsL, tvecsL =
cv2.calibrateCamera(obj_pts,img_ptsL,imgL_gray.shape[::-1],None,None)
hL,wL= imgL_gray.shape[:2]
new_mtxL, roiL= cv2.getOptimalNewCameraMatrix(mtxL,distL,(wL,hL),1,(wL,hL))

# Calibrating right camera
retR, mtxR, distR, rvecsR, tvecsR =
cv2.calibrateCamera(obj_pts,img_ptsR,imgR_gray.shape[::-1],None,None)
hR,wR= imgR_gray.shape[:2]
new_mtxR, roiR= cv2.getOptimalNewCameraMatrix(mtxR,distR,(wR,hR),1,(wR,hR))
```

We perform individual calibration of both the cameras before performing stereo calibration. But, why do we need to calibrate the cameras individually if the **stereoCalibrate()** method can also perform calibration of each of the two cameras?



The process of capturing images for stereo calibration

As there are many parameters to calculate and there is accumulation of errors in steps like corner detection and approximating points to integers. This increases the chances of the iterative method to diverge from the correct solution. So, we'll calculate the camera parameters individually and use the stereoCalibrate() method only to find the transformation between the stereo camera pair, the Essential matrix and the Fundamental matrix.

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

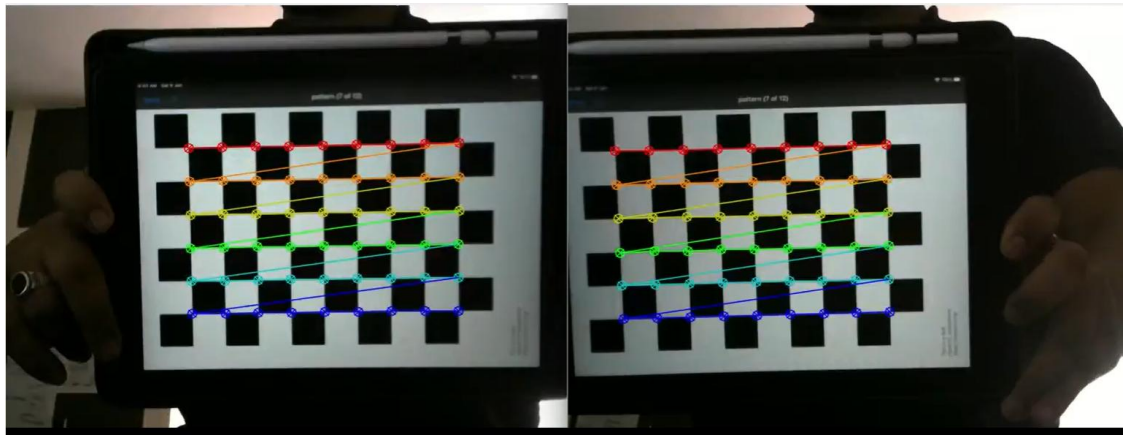
Step 2: Performing stereo calibration with fixed intrinsic parameters

As the cameras are calibrated, we pass them to the stereoCalibrate() method and set the **CALIB_FIX_INTRINSIC** flag. We also pass the 3D points and corresponding 2D pixel coordinates captured in both images.

```
flags = 0
flags |= cv2.CALIB_FIX_INTRINSIC
# Here we fix the intrinsic camara matrixes so that only Rot, Trns, Emat and Fmat are calculated.
# Hence intrinsic parameters are the same

criteria_stereo= (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)


# This step is performed to transformation between the two cameras and calculate Essential and Fundamenatl matrix
retS, new_mtxL, distL, new_mtxR, distR, Rot, Trns, Emat, Fmat = cv2.stereoCalibrate(obj_pts, img_ptsL, img_ptsR, new_mtxL, distL, new_mtxR, distR, imgL_gray.shape[::-1], criteria_stereo, flags)
```



The process of capturing images for stereo calibration

The method computes the rotation and translation between the two cameras and the Essential and Fundamental matrix.

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

Step 3: Stereo Rectification
Step 4: The undistortion of rectified stereo image pair

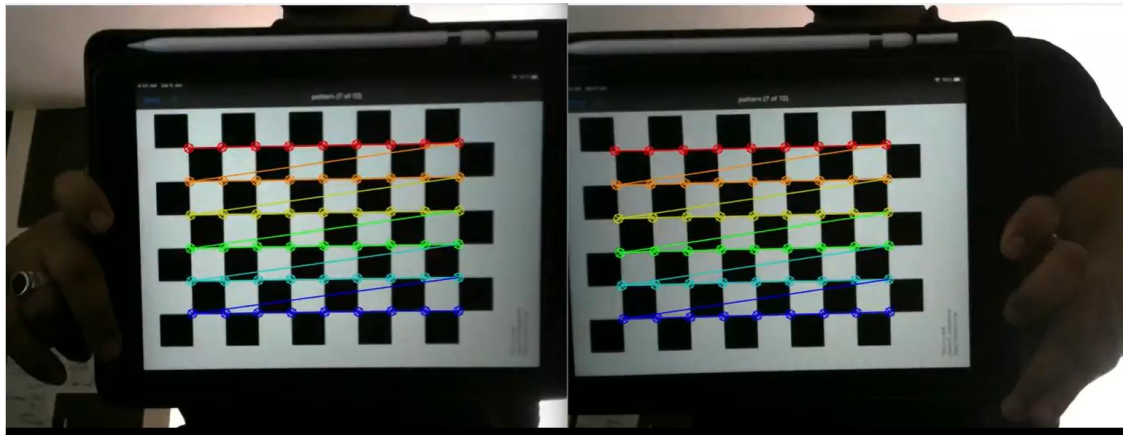
Step 3: Stereo Rectification

Using the camera intrinsics and the rotation and translation between the cameras, we can now apply stereo rectification. Stereo rectification applies rotations to make both camera image planes be in the same plane. Along with the rotation matrices, the **stereoRectify()** method also returns the projection matrices in the new coordinate space.

```
rectify_scale= 1
rect_l, rect_r, proj_mat_l, proj_mat_r, Q, roiL, roiR=
cv2.stereoRectify(new_mtxL, distL, new_mtxR, distR, imgL_gray.shape[::-1], Rot,
Trns, rectify_scale,(0,0))
```

```
Left_Stereo_Map= cv2.initUndistortRectifyMap(new_mtxL, distL, rect_l,
proj_mat_l,
                                             imgL_gray.shape[::-1],
cv2.CV_16SC2)
Right_Stereo_Map= cv2.initUndistortRectifyMap(new_mtxR, distR, rect_r,
proj_mat_r,
                                              imgR_gray.shape[::-1],
cv2.CV_16SC2)

print("Saving paraeters ......")
cv_file = cv2.FileStorage("improved_params2.xml", cv2.FILE_STORAGE_WRITE)
cv_file.write("Left_Stereo_Map_x",Left_Stereo_Map[0])
cv_file.write("Left_Stereo_Map_y",Left_Stereo_Map[1])
cv_file.write("Right_Stereo_Map_x",Right_Stereo_Map[0])
cv_file.write("Right_Stereo_Map_y",Right_Stereo_Map[1])
cv_file.release()
```



The process of capturing images for stereo calibration

Step 4: Compute the mapping required to obtain the undistorted rectified stereo image pair

As we assume that the cameras are rigidly fixed, the transformations need not be calculated again. Hence we calculate the mappings that transform a stereo image pair to an undistorted rectified stereo image pair and store them for further use.

© Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/

Disparity map building

We calculate the disparity (shift of the pixel in the two images) for each pixel and apply a proportional mapping to find the depth for a given disparity value.

```
# Reading the left and right images.

imgL = cv2.imread("../im0.png",0)
imgR = cv2.imread("../im1.png",0)

# Setting parameters for StereoSGBM algorithm
minDisparity = 0;
numDisparities = 64;
blockSize = 8;
disp12MaxDiff = 1;
uniquenessRatio = 10;
speckleWindowSize = 10;
speckleRange = 8;

# Creating an object of StereoSGBM algorithm
stereo = cv2.StereoSGBM_create(minDisparity = minDisparity,
        numDisparities = numDisparities,
        blockSize = blockSize,
        disp12MaxDiff = disp12MaxDiff,
        uniquenessRatio = uniquenessRatio,
        speckleWindowSize = speckleWindowSize,
        speckleRange = speckleRange
    )

# Calculating disparith using the StereoSGBM algorithm
disp = stereo.compute(imgL, imgR).astype(np.float32)
disp = cv2.normalize(disp,0,255,cv2.NORM_MINMAX)

# Displaying the disparity map
cv2.imshow("disparity",disp)
cv2.waitKey(0)
```



Left, right images of a real world scenario and corresponding output disparity image of a real world scenario

Disparity = x − x' = Bf/Z

where ***B*** is the baseline (Distance between the cameras), and ***f*** is the focal length, ***Z*** – is the depth

We will use a **StereoSGBM** method of OpenCV to write a code for calculating the disparity map for a given pair of images.

© Kaustubh Sadekar, Introduction to Epipolar Geometry and Stereo Vision, LearnOpenCV, 2022, https://learnopencv.com/introduction-to-epipolar-geometry-and-stereo-vision/

Quiz

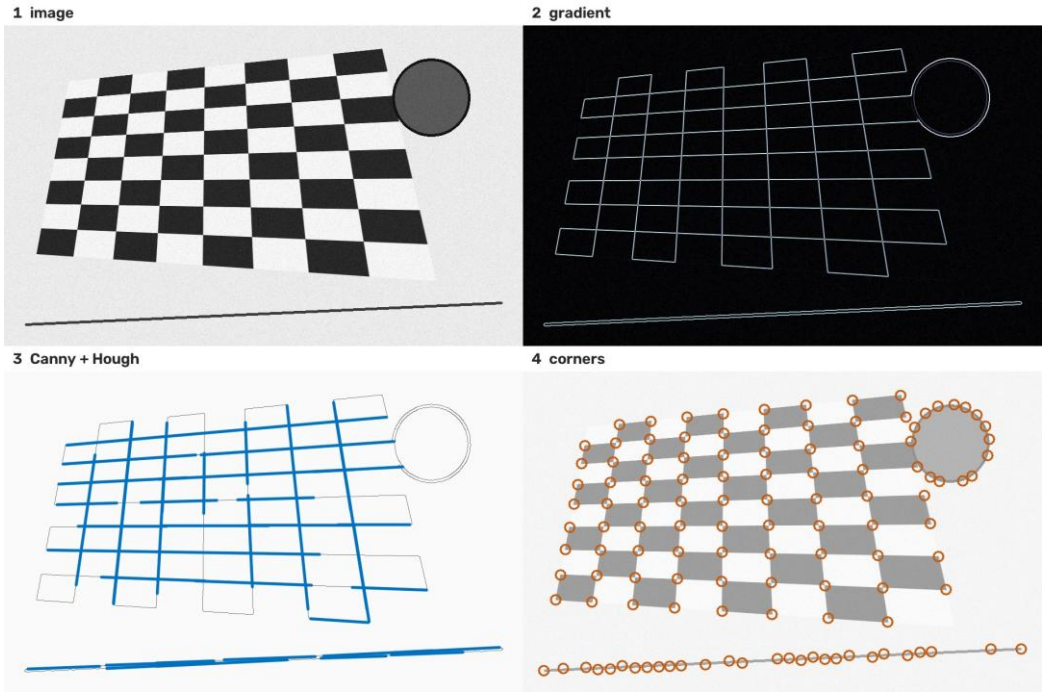
- What's the triangulation?
- What's the difference between triangulation and trilateration?
- What's the idea of Stereo Vision?
- What is the Disparity Map?
- How do we measure distances with Stereo Camera?
- What is the idea of image rectification?
- Which disparity have the close objects? (big/small?)
- What are the features of stereo camera calibration?

References

REFERENCES

1) G. Bradski and A. Kaehler, "Learning OpenCV : Computer Vision with the OpenCV Library," O'Reilly, Sebastopol, CA, 2008.
2) Stereo Calibration and Scene Reconstruction. MathWorks. http://www.mathworks.com/help/vision/examples/stereo-calibration-and-scene-reconstruction.html
3) Bouguet, J. Y. "Camera Calibration Toolbox for Matlab." Computational Vision at the California Institute of Technology. Camera Calibration Toolbox for MATLAB. http://www.vision.caltech.edu/bouguetj/calib_doc/
4) Stereo Calibration App. MathWorks. http://www.mathworks.com/help/vision/ug/stereo-camera-calibrator-app.html
5) Depth Estimation From Stereo Video http://www.mathworks.com/help/vision/examples/depth-estimation-from-stereo-video.html
6) Uncalibrated Stereo Image Rectification http://www.mathworks.com/help/vision/examples/uncalibrated-stereo-image-rectification.html
7) Kaustubh Sadekar, Making A Low-Cost Stereo Camera Using OpenCV, LearnOpenCV, 2022, https://learnopencv.com/making-a-low-cost-stereo-camera-using-opencv/
8) Kaustubh Sadekar, Introduction to Epipolar Geometry and Stereo Vision, LearnOpenCV, 2022, https://learnopencv.com/introduction-to-epipolar-geometry-and-stereo-vision/

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru